

Week12_예습과제

[통계](#) [수정](#) [삭제](#)

카사바 · 방금 전

 0

Euron_2026-1



▼ 목록 보기

8/8



07 트랜스포머

트랜스포머 모델의 주요 기능 중 하나는 기존의 순환 신경망과 같은 순차적 방식이 아닌 병렬로 입력 시퀀스를 처리하는 기능. 긴 시퀀스의 경우 트랜스포머 모델이 순환 신경망 모델보다 훨씬 빠르고 효율적으로 처리한다.

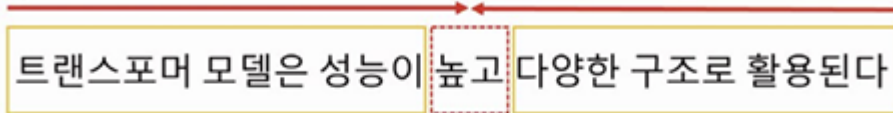
트랜스포머 모델은 기존의 순차 처리나 반복 연결에 의존하지 않고 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있도록 하는 셀프 어텐션을 기반으로 한다. 이로 인해 모델이 재귀나 합성곱 연산 없이 입력 토큰 간의 관계를 직접 모델링할 수 있다.

트랜스포머 모델 학습은 대용량 데이터셋에서 효율적이며, 데이터 양이 많은 기계 번역과 같은 작업에 적합하다. 언어 모델링 및 텍스트 분류 같은 작업에서 매우 효과적이며, 광범위한 자연어 처리 작업에서 높은 효율을 보인다.

이 장에서 소개되는 트랜스포머 기반 모델들은 오토 인코딩, 자기 회귀, 혹은 두 개의 조합으로 학습된다.

오토 인코딩 방식은 랜덤하게 문장 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업을 수행한다. 예측되는 토큰 양옆의 토큰들을 참조하기 때문에 양방향 구조를 가지며 이를 인코더라고 한다.

A) 양방향 구조



B) 단방향 구조

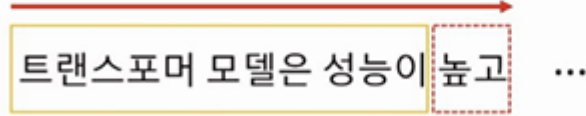


그림 7.1 트랜스포머 구조

반면에 자기 회귀 방식은 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞히는 작업을 수행한다.

예측되는 토큰의 왼쪽에 있는 토큰들만 참조할 때 단방향 구조를 가지며 이를 디코더라고 한다.

이 장에서 소개하는 트랜스포머 모델들은 양방향성 구조의 인코더(오토 인코딩) 혹은 단방향성 디코더(자기 회귀)를 사용하는 공통적 특징을 갖고 있다.

표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

Transformer

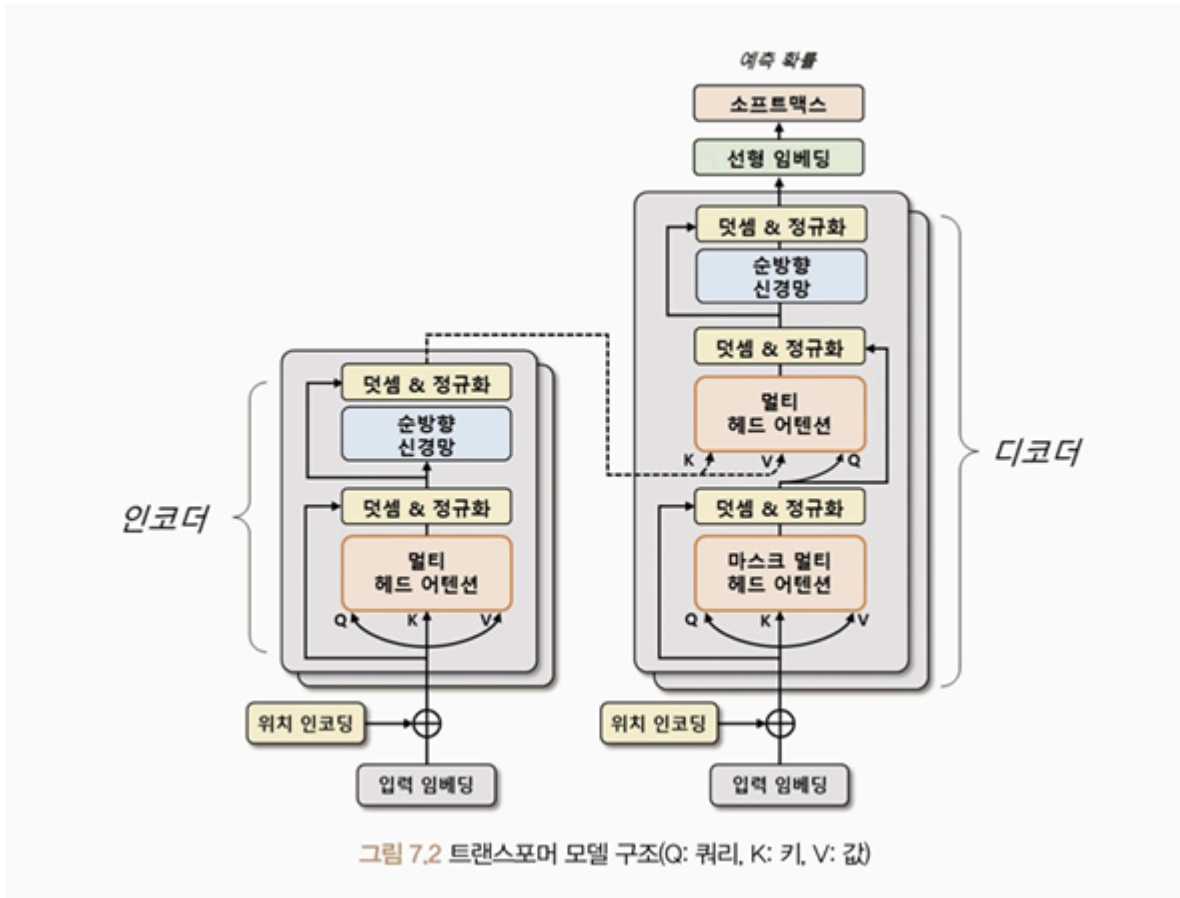
트랜스포머: 딥러닝 모델 중 하나로, 기계 번역, 챗봇, 음성 인식 등 다양한 자연어 처리 분야에서 많은 성과를 내는 모델. 어텐션 메커니즘만을 사용해 시퀀스 임베딩을 표현한다.

어텐션 메커니즘은 인코더와 디코더 간의 상호작용으로 입력 시퀀스의 중요한 부분에 초점을 맞춰 문맥을 이해하고 적절한 출력을 생성한다. 인코더는 입력 시퀀스를 임베딩하여 고차원 벡터로 변환하고, 디코더는 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성한다. 이때

어텐션 메커니즘은 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중한다. 이를 통해 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지를 파악하여 번역이나 요약문 생성과 관련된 작업 등을 수행할 수 있게 된다.

트랜스포머 모델은 기존의 순환 신경망 기반 모델보다 학습 속도가 빠르고, 병렬 처리가 가능해 대규모 데이터셋에서 높은 성능을 보인다. 또한, 임베딩 과정에서 문장의 전체 정보를 고려하기 때문에 문장 길이가 길어지더라도 성능이 유지된다.

이러한 장점으로 인해 자연어 처리 분야에서 가장 인기 있는 모델이다.



트랜스포머의 인코더와 디코더는 두 부분으로 구성돼 있으며, 각각 N개의 트랜스포머 블록으로 구성된다. 이 블록은 멀티 헤드 어텐션과 순방향 신경망으로 이루어져 있다.

멀티 헤드 어텐션: 입력 시퀀스에서 쿼리, 키, 값 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법. 이 과정에서 쿼리와 각 키의 유사도를 계산하고, 해당 유사도를 가중치로 사용해 값 벡터를 합산한다.

이렇게 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체한다. 결국 입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신한다.

순방향 신경망은 이렇게 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용된다. 순방향 신경망은 여러 개의 선형 계층으로 구성돼 있으며, 입력 벡터에 가중치를 곱하고, 편향을 더하며, 활성화 함수를 적용한다. 이 과정에서 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신된다.

트랜스포머에서는 입력 시퀀스 데이터를 source와 target 데이터로 나누어 처리한다.

e.g. 영어 -> 한글 번역하는 경우, 생성하는 언어인 한글을 타겟 데이터로 정의하고 참조하는 언어인 영어를 소스 데이터로 정의함

인코더는 소스 시퀀스 데이터를 위치 인코딩된 입력 임베딩으로 표현해 트랜스포머 블록의 출력 벡터를 생성한다. 이 출력 벡터는 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성된다.

디코더도 인코더와 유사하게 트랜스포머 블록으로 구성되어 있지만, 마스크 멀티 헤드 어텐션을 사용해 타겟 시퀀스 데이터를 순차적으로 생성시킨다. 이때 디코더 입력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보를 참조한다.

최종적으로 생성된 디코더 출력 벡터는 선형 임베딩으로 재표현되어 이미지나 자연어 모델에 활용된다. 이러한 과정을 통해 트랜스포머는 다양한 자연어 처리 작업에서 좋은 성능을 보인다.

입력 임베딩과 위치 인코딩

트랜스포머 모델에서 입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환된다.

트랜스포머는 순환 신경망과 달리 입력 시퀀스를 병렬 구조로 처리하기 때문에 단어의 순서 정보를 제공하지 않는다.

그러므로 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영해야 한다. 이를 위해 트랜스포머는 위치 인코딩 방식을 사용한다.

위치 인코딩은 입력 시퀀스의 순서 정보를 모델에 전달하는 방법이다. 각 단어의 위치 정보를 나타내는 벡터를 더하여 임베딩 벡터에 위치 정보를 반영한다.

위치 인코딩 벡터는 sin 함수와 cos 함수를 사용해 생성되며, 이를 통해 임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터를 생성한다. 위치 인코딩 벡터를 추가함으로써 모델은 단어의 순서 정보를 학습할 수 있다.

위치 인코딩은 각 토큰의 위치를 각도로 표현해 sin 함수와 cos 함수로 위치 인코딩 벡터를 계산한다.

예제 7.1 위치 인코딩

```
import math
import torch
```

```

from torch import nn
from matplotlib import pyplot as plt

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

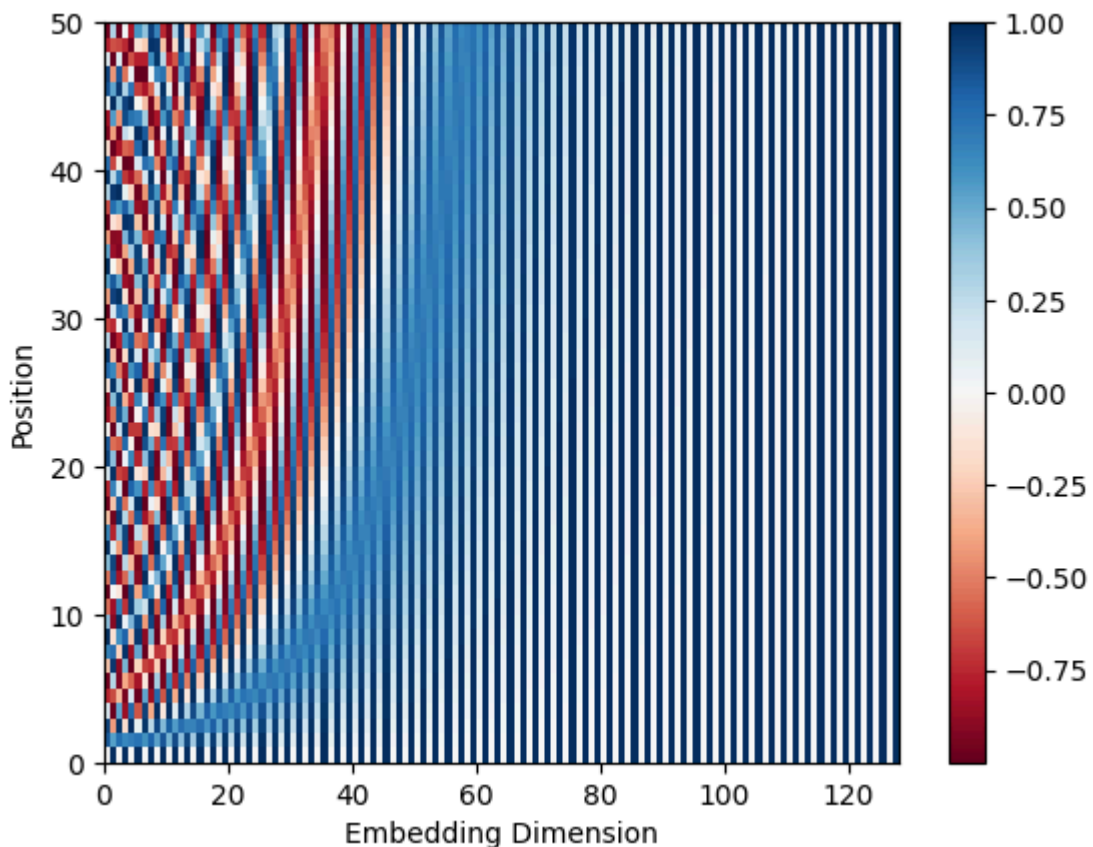
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[: x.size(0)]
        return self.dropout(x)

encoding = PositionalEncoding(d_model=128, max_len=50)

plt.pcolormesh(encoding.pe.numpy().squeeze(), cmap="RdBu")
plt.xlabel("Embedding Dimension")
plt.xlim((0, 128))
plt.ylabel("Position")
plt.colorbar()
plt.show()

```



PositionalEncoding 클래스는 입력 임베딩 차원(d_{model})과 최대 시퀀스(max_len)를 입력받는다. 입력 시퀀스의 위치마다 $\sin$ 과 $\cos$ 함수로 위치 인코딩을 계산한다.

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

pos 는 입력 시퀀스에서 해당 단어의 위치를 나타내며, i 는 임베딩 벡터의 차원 인덱스를 의미한다. 차원 인덱스가 짝수면 $\sin$ 함수 수식을 적용하며, 홀수면 $\cos$ 함수 수식을 적용한다.

pe 변수의 텐서 차원은 $[50, 1, 128]$ 이 되며, $[최대\ 시퀀스, 1, 입력\ 임베딩\ 차원]$ 을 의미한다. 출력 결과를 확인해 보면 위치별 임베딩 차원이 주기적인 값으로 구성되는 것을 확인할 수 있다. 산출된 위치 인코딩은 입력 임베딩과 더해진 후 드롭아웃이 적용된다.

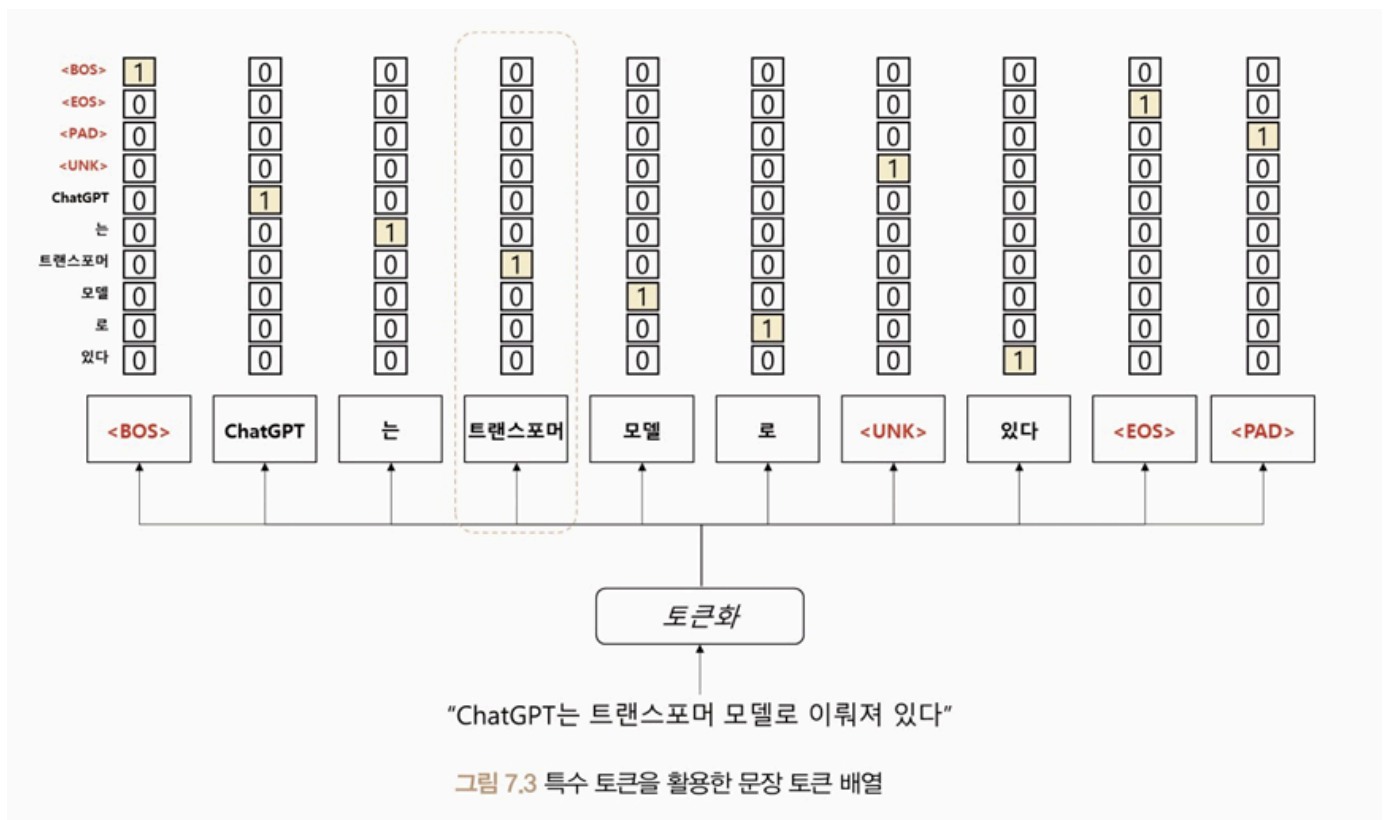
예제에 적용한 `register_buffer` 메서드는 모델이 매개변수를 갱신하지 않도록 설정한다.

특수 토큰

트랜스포머는 단어 토큰 이외의 특수 토큰을 활용하여 문장을 표현한다. 이 특수 토큰은 입력 시퀀스의 시작과 끝을 나타내거나 마스킹 영역으로 사용된다.

특수 토큰으로 모델이 입력 시퀀스의 시작과 끝을 인식할 수 있게 하며, 마스킹을 통해 일부 입력을 무시할 수 있다.

e.g. 번역 모델에서는 디코더의 입력 시퀀스에서 현재 위치 이후의 토큰을 마스크해 이전 토큰만을 참조한다.

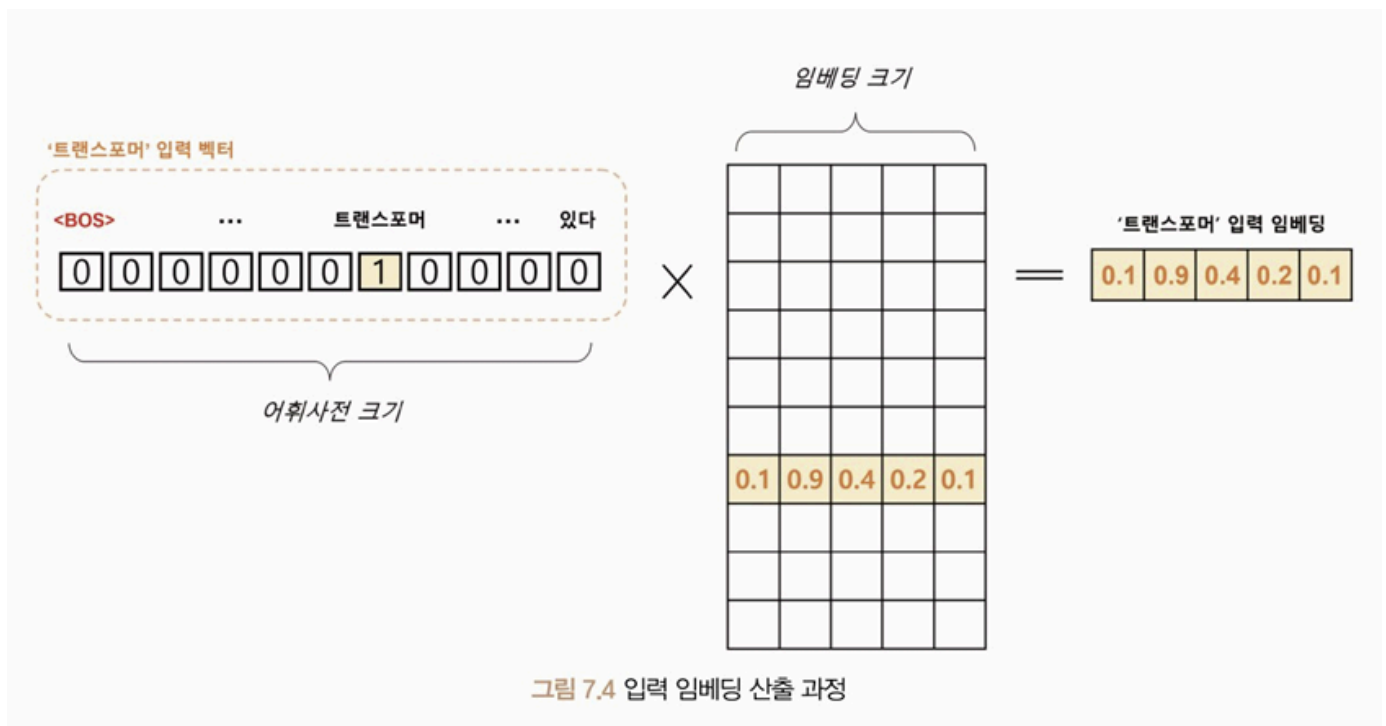


그림에서 사용한 BOS(Beginning of Sentence), EOS(End of Sentence), UNK(Unknown), PAD(Padding) 토큰은 모두 특수 토큰으로 자연어 처리에서 일반적으로 사용된다.

UNK: 어휘 사전에 없는 단어로, 모르는 단어 의미. 모델이 이전에 본 적 없는 단어를 처리할 때 사용됨

PAD: 모든 문장을 일정한 길이로 맞추기 위해 사용됨. 짧은 문장의 빈 공간 채울 때 사용

이렇게 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현한다.



어휘 사전 크기를 V , 입력 임베딩 차원을 d 라고 했을 때, '트랜스포머'의 원-핫 벡터는 $[1, V]$ 크기를 갖는다. 이 원-핫 벡터는 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기 벡터로 변환된다.

N 개의 문장이 최대 S 개의 토큰 길이를 가질 때 $[N, S, V]$ 크기의 원-핫 벡터 텐서는 $[N, S, d]$ 크기의 임베딩 텐서로 변환된다.

이 임베딩 텐서는 입력으로 사용되며, 트랜스포머의 모든 계층에서 공유되는 텐서로 사용된다.

트랜스포머 인코더

트랜스포머 인코더: 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행한다. 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성되며, 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달한다.

인코더 계층에서 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해준다. 그리고 산출된 인코더 계층 출력은 디코더 계층으로 전달된다.

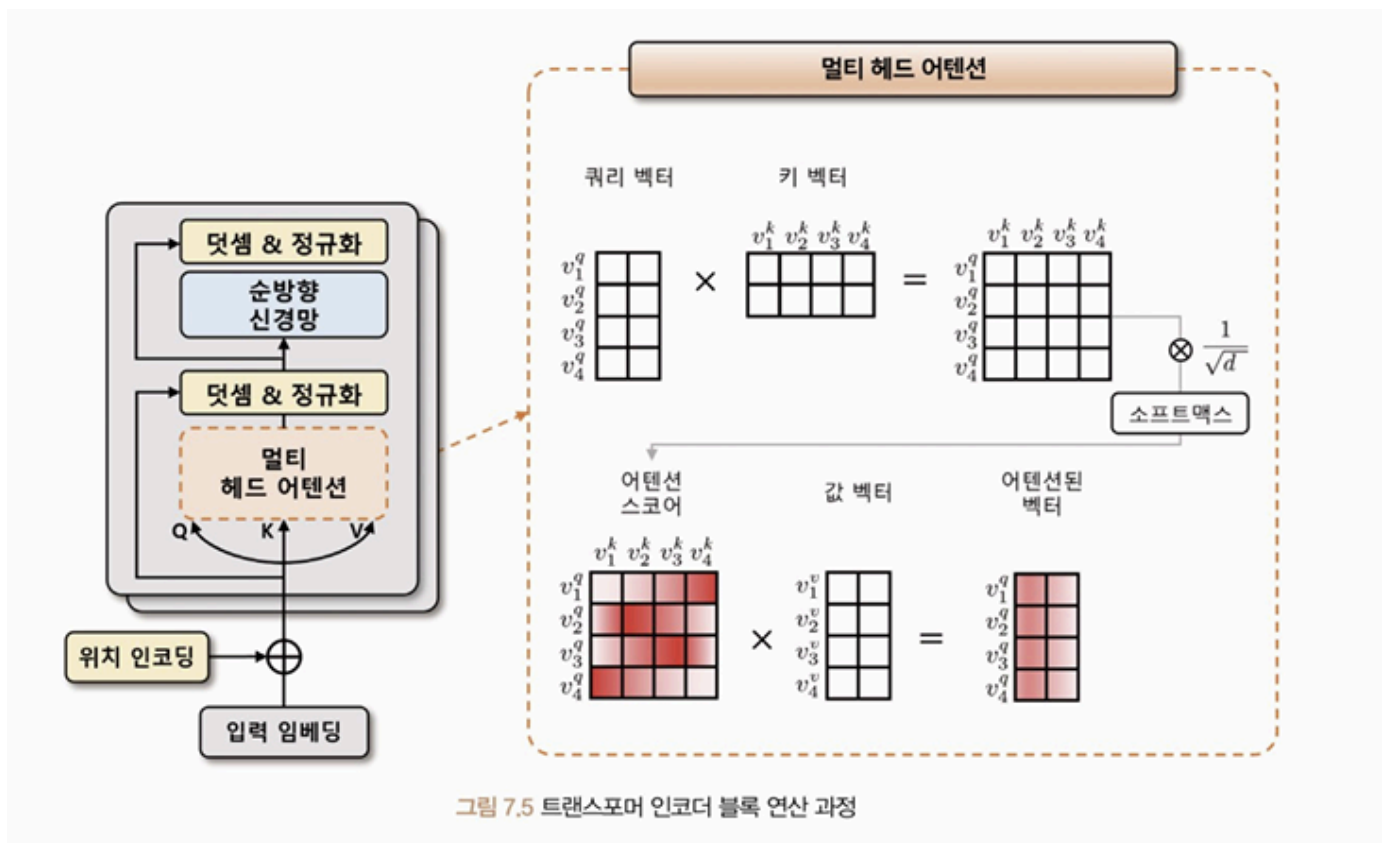


그림 7.5 트랜스포머 인코더 블록 연산 과정

트랜스포머 인코더는 위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력받는다. 멀티 헤드 어텐션 단계에서 입력 텐서 차원이 $[N, s, d]$ 라고 한다면 입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터를 생성한다. 생성된 3개의 벡터는 각각 쿼리(Q), 키(K), 값(V) 벡터라고 정의한다.

수식 7.2 어텐션 스코어 계산식

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

셀프 어텐션 과정에서는 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구하고 이 스코어값에 벡터 차원의 제곱근만큼 나눠 보정한다. 이 보정값은 벡터 차원이 커질 때 스코어값이 같이 커지는 문제를 완화하기 위해 적용한다.

멀티 헤드는 이러한 셀프 어텐션을 여러 번 수행해 여러 개의 헤드를 만든다. 각각의 헤드가 독립적으로 어텐션을 수행하고 그 결과를 합친다.

순방향 신경망은 두 가지 방법을 적용할 수 있는데, 선형 임베딩과 ReLU로 이뤄진 인공 신경망이나 1차원 합성곱이 사용된다. 이어서 다시 닷셈 & 정규화 과정이 수행된다.

트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성된다. 이전 블록에서 출력된 벡터는 다음 블록의 입력으로 전달되어 인코더 블록을 통과하면서 점차 입력 시퀀스의 정보가 추상화된다.

트랜스포머 디코더

트랜스포머 디코더: 위치 인코딩이 적용된 타깃 데이터의 입력 임베딩을 입력받는다.

위치 인코딩은 입력 시퀀스 내에서 각 단어의 상대적인 위치 정보를 전달하는 기법으로 디코더의 입력 임베딩에 위치 정보를 추가함으로써 디코더가 입력 시퀀스의 순서 정보를 학습할 수 있게 된다.

트랜스포머 모델에서 인코더의 멀티 헤드 어텐션 모듈은 인과성을 반영한 마스크 멀티 헤드 어텐션 모듈로 대체된다.

마스크를 적용하면 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 되며, 인과성이 보장된다.

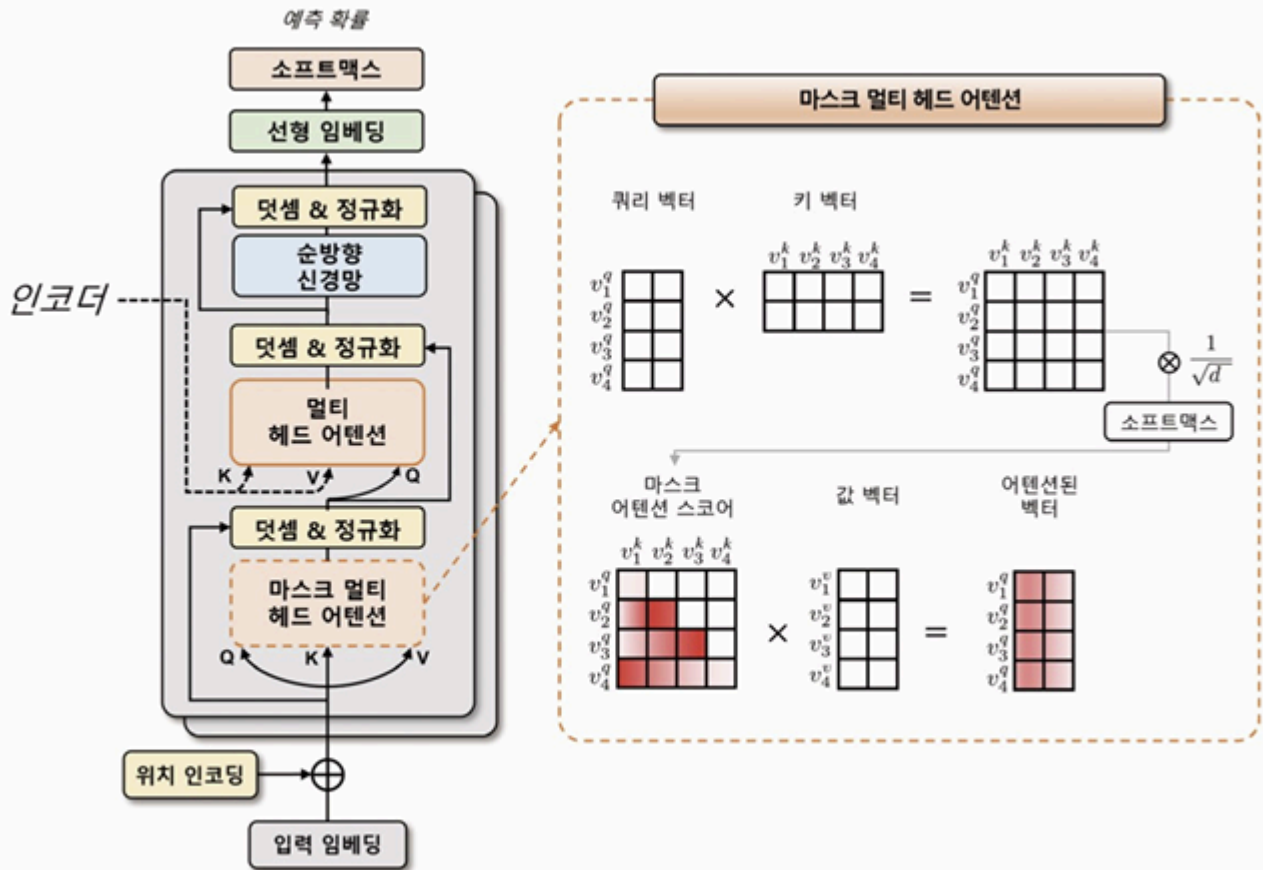


그림 7.6 트랜스포머 디코더 블록 연산 과정

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

모델 실습

Multi30k 데이터셋은 영어-독일어 병렬 말뭉치로 약 30,000개의 데이터를 제공하며 토치 데이터와 토치 텍스트 라이브러리로 해당 데이터셋을 쉽게 다운받을 수 있다.

포르타락커 라이브러리는 파이썬에서 파일 락을 관리하기 위한 라이브러리로, 파일 락을 사용해 여러 프로세스 간에 동시에 파일을 수정하거나 읽는 것을 방지한다. 포르타락커는 Multi30k 데이터셋을 다운로드하고 압축을 해제하는 과정에서 내부적으로 사용된다.

예제 7.2 데이터셋 다운로드 및 전처리

```
pip install datasets

!pip install datasets spacy
!python -m spacy download de_core_news_sm
!python -m spacy download en_core_web_sm

from datasets import load_dataset
import spacy
from collections import Counter

# — 기본 설정 —————
SRC_LANGUAGE = "de"
TGT_LANGUAGE = "en"
UNK_IDX, PAD_IDX, BOS_IDX, EOS_IDX = 0, 1, 2, 3
special_symbols = ["<unk>", "<pad>", "<bos>", "<eos>"]

# — 토큰나이저 —————
spacy_de = spacy.load("de_core_news_sm")
spacy_en = spacy.load("en_core_web_sm")

token_transform = {
    SRC_LANGUAGE: lambda text: [tok.text for tok in spacy_de.tokenizer(text)],
    TGT_LANGUAGE: lambda text: [tok.text for tok in spacy_en.tokenizer(text)],
}

# — 데이터셋 —————
dataset = load_dataset("bentrevett/multi30k")
train_data = dataset["train"]

# — Vocab 클래스 (torchtext.vocab 대체) —————
class Vocab:
    def __init__(self, counter, specials, min_freq=1):
        self.itos = specials.copy() # index → token
        for token, freq in counter.items():
            if freq >= min_freq and token not in specials:
                self.itos.append(token)
        self.stoi = {tok: i for i, tok in enumerate(self.itos)} # token → index
        self.default_index = 0 # <unk>

    def __len__(self):
        return len(self.itos)

    def __getitem__(self, token):
```

```

        return self.stoi.get(token, self.default_index)

    def lookup_indices(self, tokens):
        return [self[tok] for tok in tokens]

    def lookup_tokens(self, indices):
        return [self.itos[i] for i in indices]

    def set_default_index(self, idx):
        self.default_index = idx

# — Vocab 빌드 —————
vocab_transform = {}
for lang in [SRC_LANGUAGE, TGT_LANGUAGE]:
    counter = Counter()
    for item in train_data:
        counter.update(token_transform[lang](item[lang]))
    vocab_transform[lang] = Vocab(counter, special_symbols, min_freq=1)

print("DE vocab size:", len(vocab_transform["de"]))
print("EN vocab size:", len(vocab_transform["en"]))

```

출력 결과

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md:
 1.15k/? [00:00<00:00, 30.6kB/s]
train.jsonl:
 4.60M/? [00:00<00:00, 95.2MB/s]
val.jsonl:
 164k/? [00:00<00:00, 11.7MB/s]
test.jsonl:
 156k/? [00:00<00:00, 12.4MB/s]
Generating train split: 100%
 29000/29000 [00:00<00:00, 401623.26 examples/s]
Generating validation split: 100%
 1014/1014 [00:00<00:00, 52229.21 examples/s]
Generating test split: 100%
 1000/1000 [00:00<00:00, 62255.89 examples/s]
DE vocab size: 19214
EN vocab size: 10837

```

예제 7.3 트랜스포머 모델 구성

```

import math
import torch
from torch import nn

```

```

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[: x.size(0)]
        return self.dropout(x)

class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size, emb_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_size)
        self.emb_size = emb_size

    def forward(self, tokens):
        return self.embedding(tokens.long()) * math.sqrt(self.emb_size)

class Seq2SeqTransformer(nn.Module):
    def __init__(
        self,
        num_encoder_layers,
        num_decoder_layers,
        emb_size,
        max_len,
        nhead,
        src_vocab_size,
        tgt_vocab_size,
        dim_feedforward,
        dropout=0.1,
    ):
        super().__init__()
        self.src_tok_emb = TokenEmbedding(src_vocab_size, emb_size)
        self.tgt_tok_emb = TokenEmbedding(tgt_vocab_size, emb_size)
        self.positional_encoding = PositionalEncoding(
            d_model=emb_size, max_len=max_len, dropout=dropout
        )
        self.transformer = nn.Transformer(
            d_model=emb_size,
            nhead=nhead,
            num_encoder_layers=num_encoder_layers,
            num_decoder_layers=num_decoder_layers,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
        )
        self.generator = nn.Linear(emb_size, tgt_vocab_size)

```

```

def forward(
    self,
    src,
    trg,
    src_mask,
    tgt_mask,
    src_padding_mask,
    tgt_padding_mask,
    memory_key_padding_mask,
):
    src_emb = self.positional_encoding(self.src_tok_emb(src))
    tgt_emb = self.positional_encoding(self.tgt_tok_emb(trg))
    outs = self.transformer(
        src=src_emb,
        tgt=tgt_emb,
        src_mask=src_mask,
        tgt_mask=tgt_mask,
        memory_mask=None,
        src_key_padding_mask=src_padding_mask,
        tgt_key_padding_mask=tgt_padding_mask,
        memory_key_padding_mask=memory_key_padding_mask
    )
    return self.generator(outs)

def encode(self, src, src_mask):
    return self.transformer.encoder(
        self.positional_encoding(self.src_tok_emb(src)), src_mask
    )

def decode(self, tgt, memory, tgt_mask):
    return self.transformer.decoder(
        self.positional_encoding(self.tgt_tok_emb(tgt)), memory, tgt_mask
    )

```

트랜스포머 클래스

```

transformer = torch.nn.Transformer(
    d_model=512,
    nhead=8,
    num_encoder_layers=6,
    num_decoder_layers=6,
    dim_feedforward=2048,
    dropout=0.1,
    activation=torch.nn.functional.relu,
    layer_norm_eps=1e-05,
)

```

임베딩 차원(d_model): 트랜스포머 모델의 입력과 출력 차원의 크기를 정의한다. 임베딩 차원 크기와 동일한 값

헤드(nhead): 멀티 헤드 어텐션 헤드의 개수 정의. 헤드의 개수는 모델이 어텐션을 수행하는 방법을 결정하며, 헤드의 개수가 많을수록 모델의 병렬 처리 능력이 증가한다. 단, 헤드의 개수가 증가할수록 모델 매개변수 수도 증가한다.

인코더 계층 개수(num_encoder_layers), 디코더 계층 개수(num_decoder_layers): 인코더와 디코더의 계층 수. 모델의 복잡도와 성능에 영향을 미친다. 계층 개수가 많을수록 더 복잡한 문제를 해결할 수 있지만, 너무 많을 경우 과대적합될 수 있다.

순방향 신경망 크기(dim_feedforward): 순방향 신경망의 은닉층 크기 정의. 모델의 복잡도와 성능에 영향을 미친다.

드롭아웃(dropout): 인코더와 디코더 계층에 적용되는 드롭아웃 비율을 적용함

활성화 함수(activation): 순방향 신경망에 적용되는 활성화 함수

계층 정규화 입실론(layer_norm_eps): 계층 정규화를 수행할 때 분모에 더해지는 입실론 값

트랜스포머 순방향 메서드

```
output = transformer.forward(  
    src,  
    tgt,  
    src_mask=None,  
    tgt_mask=None,  
    memory_mask=None,  
    src_key_padding_mask=None,  
    tgt_key_padding_mask=None,  
    memory_key_padding_mask=None,  
)
```

소스(src), 타겟(tgt): 인코더와 디코더에 대한 시퀀스

소스 마스크(src_mask), 타겟 마스크(tgt_mask): 소스와 타겟 시퀀스의 마스크

마스크 값이 0이면 해당 위치에서는 모든 입력 단어가 동일한 가중치를 갖고 어텐션이 수행되며, 1이면 모든 입력 단어의 가중치가 0으로 설정돼 어텐션 연산이 수행되지 않는다.

마스크 값이 $-\infty$ 면 해당 위치에서는 어텐션 연산 결과에 0으로 가중치가 부여돼 마스킹된 위치의 정보를 모델이 무시하게 만든다.

$+\infty$ 로 입력하면 모든 입력 단어에 무한대의 가중치가 부여된다.

메모리 마스크(memory_mask): 인코더 출력의 마스크.

소스, 타겟, 메모리 키 패딩 마스크(key_padding_mask): 소스, 타겟, 메모리 시퀀스에 대한 패딩 마스크

예제 7.4 트랜스포머 모델 구조

```
from torch import optim

BATCH_SIZE = 128
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

model = Seq2SeqTransformer(
    num_encoder_layers=3,
    num_decoder_layers=3,
    emb_size=512,
    max_len=512,
    nhead=8,
    src_vocab_size=len(vocab_transform[SRC_LANGUAGE]),
    tgt_vocab_size=len(vocab_transform[TGT_LANGUAGE]),
    dim_feedforward=512,
).to(DEVICE)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX).to(DEVICE)
optimizer = optim.Adam(model.parameters())

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("└", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("│ └", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("│ │ └", sssub_name)
```

출력 결과

```
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/transformer.py:144: UserWarning: enable
self.encoder = TransformerEncoder(
src_tok_emb
└ embedding
tgt_tok_emb
└ embedding
positional_encoding
└ dropout
transformer
└ encoder
```

```

|   L layers
|   |   L 0
|   |   L 1
|   |   L 2
|   L norm
L decoder
|   L layers
|   |   L 0
|   |   L 1
|   |   L 2
|   L norm
generator

```

인코더와 디코더가 각각 세 개(0, 1, 2)의 계층으로 구성된다.

손실 함수는 교차 엔트로피 함수를 적용한다. 무시되는 색인(ignore_index) 값을 패딩 토큰 (PAD_IDX)을 할당해 모델이 학습하는 동안 무시해야 할 클래스 레이블을 지정한다.

예제 7.5 배치 데이터 생성

```

from torch.utils.data import DataLoader
from torch.nn.utils.rnn import pad_sequence
from datasets import load_dataset
import torch

def sequential_transforms(*transforms):
    def func(txt_input):
        for transform in transforms:
            txt_input = transform(txt_input)
        return txt_input
    return func

def input_transform(token_ids):
    return torch.cat(
        (torch.tensor([BOS_IDX]), torch.tensor(token_ids), torch.tensor([EOS_IDX]))
    )

def collator(batch):
    src_batch, tgt_batch = [], []
    for src_sample, tgt_sample in batch:
        src_batch.append(text_transform[SRC_LANGUAGE](src_sample.rstrip("\n")))
        tgt_batch.append(text_transform[TGT_LANGUAGE](tgt_sample.rstrip("\n")))

    src_batch = pad_sequence(src_batch, padding_value=PAD_IDX)
    tgt_batch = pad_sequence(tgt_batch, padding_value=PAD_IDX)
    return src_batch, tgt_batch

text_transform = {}
for language in [SRC_LANGUAGE, TGT_LANGUAGE]:
    text_transform[language] = sequential_transforms(
        token_transform[language],
        vocab_transform[language].lookup_indices, # ← 이것도 수정
        input_transform
    )

```

```
# — Multi30k → datasets로 대체 —————
dataset = load_dataset("bentrevett/multi30k")
valid_data = [(item["de"], item["en"]) for item in dataset["validation"]]

dataloader = DataLoader(valid_data, batch_size=BATCH_SIZE, collate_fn=collator)
source_tensor, target_tensor = next(iter(dataloader))

print("(source, target):")
print(valid_data[0])

print("source_batch:", source_tensor.shape)
print(source_tensor)

print("target_batch:", target_tensor.shape)
print(target_tensor)
```

출력 결과

```
(source, target):
('Eine Gruppe von Männern lädt Baumwolle auf einen Lastwagen', 'A group of men are loading cot
source_batch: torch.Size([35, 128])
tensor([[ 2,  2,  2, ...,  2,  2,  2],
        [ 70, 23, 23, ..., 23,  4, 23],
        [130, 30, 250, ..., 30, 4360, 24],
        ...,
        [ 1,  1,  1, ...,  1,  1,  1],
        [ 1,  1,  1, ...,  1,  1,  1],
        [ 1,  1,  1, ...,  1,  1,  1]])
target_batch: torch.Size([30, 128])
tensor([[ 2,  2,  2, ...,  2,  2,  2],
        [ 25, 25, 25, ..., 1244,  4, 25],
        [255, 32, 246, ..., 32, 3035, 26],
        ...,
        [ 1,  1,  1, ...,  1,  1,  1],
        [ 1,  1,  1, ...,  1,  1,  1],
        [ 1,  1,  1, ...,  1,  1,  1]])
```

sequential_transforms: 여러 개의 전처리 함수를 인자로 받아 이를 차례로 적용하는 함수를 반환하는 함수

collator 함수는 배치 단위로 데이터를 처리한다.

예제 7.6 어텐션 마스크 생성

```
def generate_square_subsequent_mask(s):
    mask = (torch.triu(torch.ones((s, s), device=DEVICE)) == 1).transpose(0, 1)
    mask = (
        mask.float()
        .masked_fill(mask == 0, float("-inf"))
        .masked_fill(mask == 1, float(0.0))
    )
```



```
[False, False, False, ..., True, True, True],
[False, False, False, ..., True, True, True]])
```

source_mask: 셀프 어텐션 과정에서 참조되는 소스 데이터의 시퀀스 범위. False인 위치는 셀프 어텐션에 참조되는 토큰이 되며, True인 위치는 어텐션에서 제외되는 토큰이 된다.

예제 7.7 모델 학습 및 평가

```
def run(model, optimizer, criterion, split):
    model.train() if split == "train" else model.eval()

    # — Multi30k → datasets로 대체 —————
    split_map = {"train": "train", "valid": "validation"}
    data = load_dataset("bentrevett/multi30k")[split_map[split]]
    data_iter = [(item["de"], item["en"]) for item in data]
    dataloader = DataLoader(data_iter, batch_size=BATCH_SIZE, collate_fn=collator)
    # —————

    losses = 0
    for source_batch, target_batch in dataloader:
        source_batch = source_batch.to(DEVICE)
        target_batch = target_batch.to(DEVICE)

        target_input = target_batch[:-1, :]
        target_output = target_batch[1:, :]

        src_mask, tgt_mask, src_padding_mask, tgt_padding_mask = create_mask(
            source_batch, target_input
        )

        logits = model(
            src=source_batch,
            trg=target_input,
            src_mask=src_mask,
            tgt_mask=tgt_mask,
            src_padding_mask=src_padding_mask,
            tgt_padding_mask=tgt_padding_mask,
            memory_key_padding_mask=src_padding_mask,
        )

        optimizer.zero_grad()
        loss = criterion(logits.reshape(-1, logits.shape[-1]), target_output.reshape(-1))
        if split == "train":
            loss.backward()
            optimizer.step()
        losses += loss.item()

    return losses / len(dataloader)

for epoch in range(5):
    train_loss = run(model, optimizer, criterion, "train")
```

```
val_loss = run(model, optimizer, criterion, "valid")
print(f"Epoch: {epoch+1}, Train loss: {train_loss:.3f}, Val loss: {val_loss:.3f}")
```

출력 결과

```
Epoch: 1, Train loss: 4.845, Val loss: 4.107
Epoch: 2, Train loss: 3.846, Val loss: 3.719
Epoch: 3, Train loss: 3.526, Val loss: 3.600
Epoch: 4, Train loss: 3.367, Val loss: 3.586
Epoch: 5, Train loss: 3.238, Val loss: 3.555
```

예제 7.8 트랜스포머 모델 번역 결과

```
def greedy_decode(model, source_tensor, source_mask, max_len, start_symbol):
    source_tensor = source_tensor.to(DEVICE)
    source_mask = source_mask.to(DEVICE)

    memory = model.encode(source_tensor, source_mask)
    ys = torch.ones(1, 1).fill_(start_symbol).type(torch.long).to(DEVICE)
    for i in range(max_len - 1):
        memory = memory.to(DEVICE)
        target_mask = generate_square_subsequent_mask(ys.size(0))
        target_mask = target_mask.type(torch.bool).to(DEVICE)

        out = model.decode(ys, memory, target_mask)
        out = out.transpose(0, 1)
        prob = model.generator(out[:, -1])
        _, next_word = torch.max(prob, dim=1)
        next_word = next_word.item()

        ys = torch.cat(
            [ys, torch.ones(1, 1).type_as(source_tensor.data).fill_(next_word)], dim=0
        )
        if next_word == EOS_IDX:
            break
    return ys

def translate(model, source_sentence):
    model.eval()
    source_tensor = text_transform[SRC_LANGUAGE](source_sentence).view(-1, 1)
    num_tokens = source_tensor.shape[0]
    src_mask = (torch.zeros(num_tokens, num_tokens)).type(torch.bool)
    tgt_tokens = greedy_decode(
        model, source_tensor, src_mask, max_len=num_tokens + 5, start_symbol=BOS_IDX
    ).flatten()
    output = vocab_transform[TGT_LANGUAGE].lookup_tokens(list(tgt_tokens.cpu().numpy()))[1:-1]
    return " ".join(output)
```

```
output_oov = translate(model, "Eine Gruppe von Menschen steht vor einem Iglu .")
output = translate(model, "Eine Gruppe von Menschen steht vor einem Gebäude .")
```

```
print(output_oov)
print(output)
```

출력 결과

```
A group of people are sitting on a bench .
A group of people are sitting on a bench .
```

모델 번역 방식은 그리드 디코딩 방식으로 번역 결과를 출력한다. 그리드 디코딩은 디코더 네트워크가 생성한 확률 분포에서 가장 높은 확률을 가지는 단어를 선택하는 방법으로, 현재 시점에서 가장 확률이 높은 단어를 선택하여 디코딩을 진행한다.

GPT

GPT: 2018년 OpenAI에서 발표한 트랜스포머 기반 언어 모델로 가장 널리 알려진 언어 모델

GPT 모델은 트랜스포머 디코더를 여러 층 쌓아 만든 언어 모델이다. 트랜스포머 인코더는 입력 문장의 특징을 추출하는 데 초점을 두고 있다면, 디코더는 입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장을 생성하는 데 초점을 두고 있다.

이런 특성 때문에 언어 모델링 작업에서 더 높은 성능을 발휘한다. GPT 모델은 대규모 텍스트 데이터셋으로 사전 학습해 초기화되며, 특정 자연어 처리 작업에 맞게 미세 조정해 사용한다.

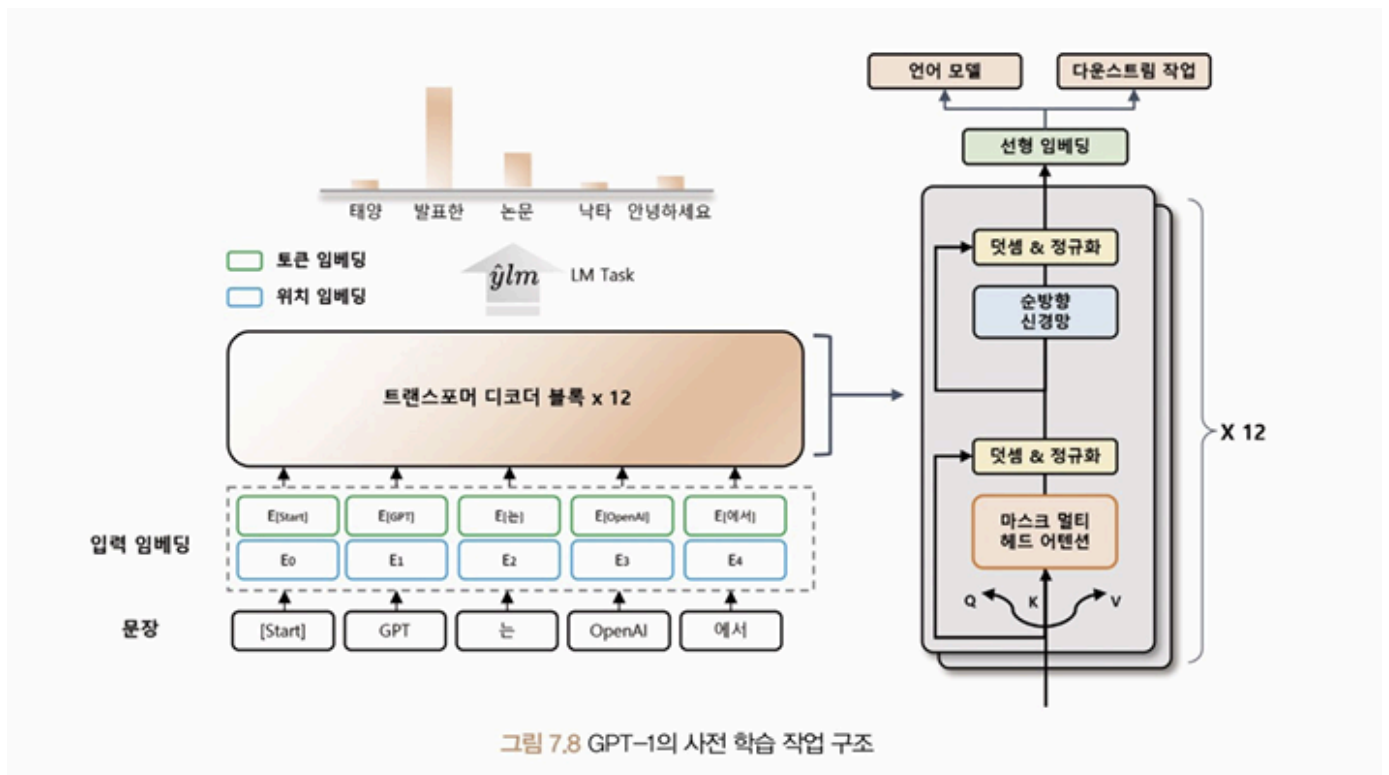
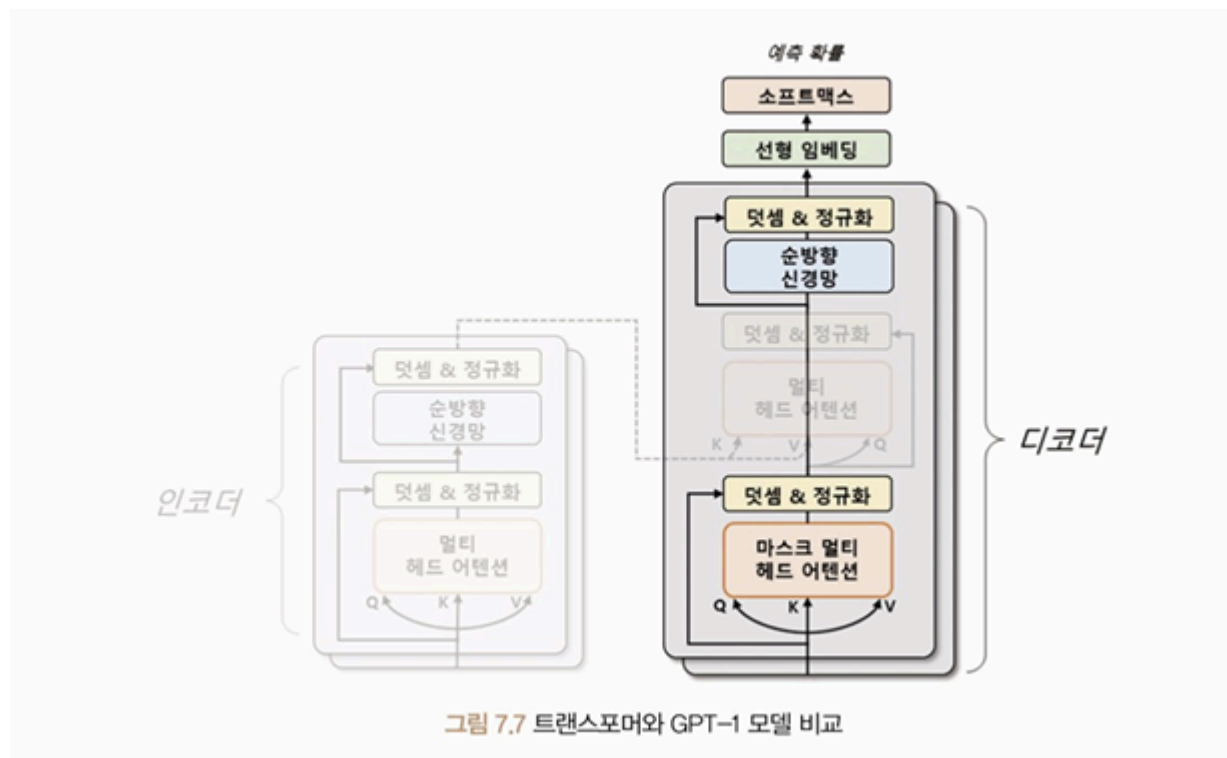
GPT는 일반적으로 예측 성능이 뛰어나고 적은 데이터로도 높은 성능을 보인다.

많은 양의 매개변수를 줄이기 위해 OpenAI는 RLHF 방법을 도입했다. RLHF란 인간의 피드백을 사용하는 강화 학습 방법이다. 이 방법은 인간이 모델이 생성한 결과물을 평가하고, 모델이 좋은 결과물을 생성하면 보상을 주어서 모델을 학습시킨다.

GPT-1

GPT-1: 트랜스포머 구조를 바탕으로 한 단방향 언어 모델. 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측하는 모델

이전 단어들을 차례대로 입력하면서 다음 단어를 예측하는 방식으로 동작한다. 모델의 계산량이 줄어들기 때문에 학습 속도가 빠르고, 모델의 크기가 작아질 수 있다는 장점이 있다.



GPT-2

GPT-1은 입력 문장을 최대 512의 길이만큼 받아들여 다음 토큰을 예측했지만, GPT-2에서는 1,024의 길이까지 입력을 처리할 수 있다. 이외에도 12개의 디코더 층을 사용한 GPT-1과 달리,

48개의 디코더 계층을 사용해 이전 모델인 GPT-1보다 복잡한 패턴을 학습할 수 있게 됐다.

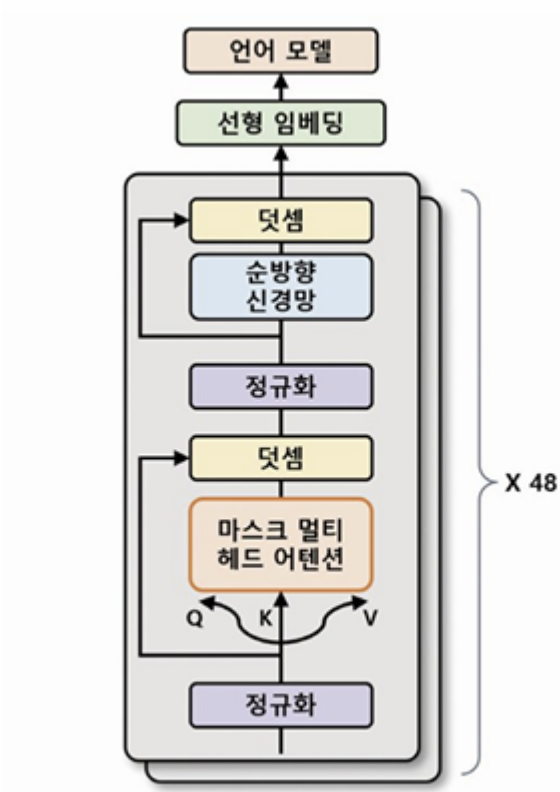


그림 7.10 GPT-2의 모델 구조

GPT-2는 GPT-1보다 훨씬 더 많은 양의 데이터를 이용해 사전 학습됐다.

GPT-3

GPT-3는 몇몇 모델 매개변수 변화를 통해 GPT-2보다 약 116배 많은 매개변수를 가진 모델이 됐다.

GPT-3는 규모가 큰 모델로 96개의 헤드를 가진 멀티 헤드 어텐션을 사용하며, 트랜스포머 디코더 층도 96개의 층을 사용한다.

질의 응답

입력 프롬프트

문서 : 생성적 사전학습 트랜스포머(Generative Pre-Trained Transformer 3), GPT-3는 OpenAI에서 만든 대형 언어 모델이다.

질문 : GPT-3를 만든 곳은 어디인가?

답변 :

OpenAI

출력값

자연어 추론

입력 프롬프트

자연어 추론 : OpenAI는 미국의 인공지능 연구소로, 인류에게 이익을 주는 것을 목표로 한다. GPT-3등 거대 언어 모델을 개발하여 많은 이목을 끌었다.

질문 : OpenAI는 상장사인가? 참, 거짓, 둘 다 아님

답변 :

둘 다 아님

출력값

수학 문제 풀이

입력 프롬프트

질문 : $(2 * 4) * 6$ 은 무엇인가?

답변 :

48

출력값

일반 상식 질의 응답

입력 프롬프트

질문 : 물은 섭씨 몇 도에서 어는가?

답변 :

0도

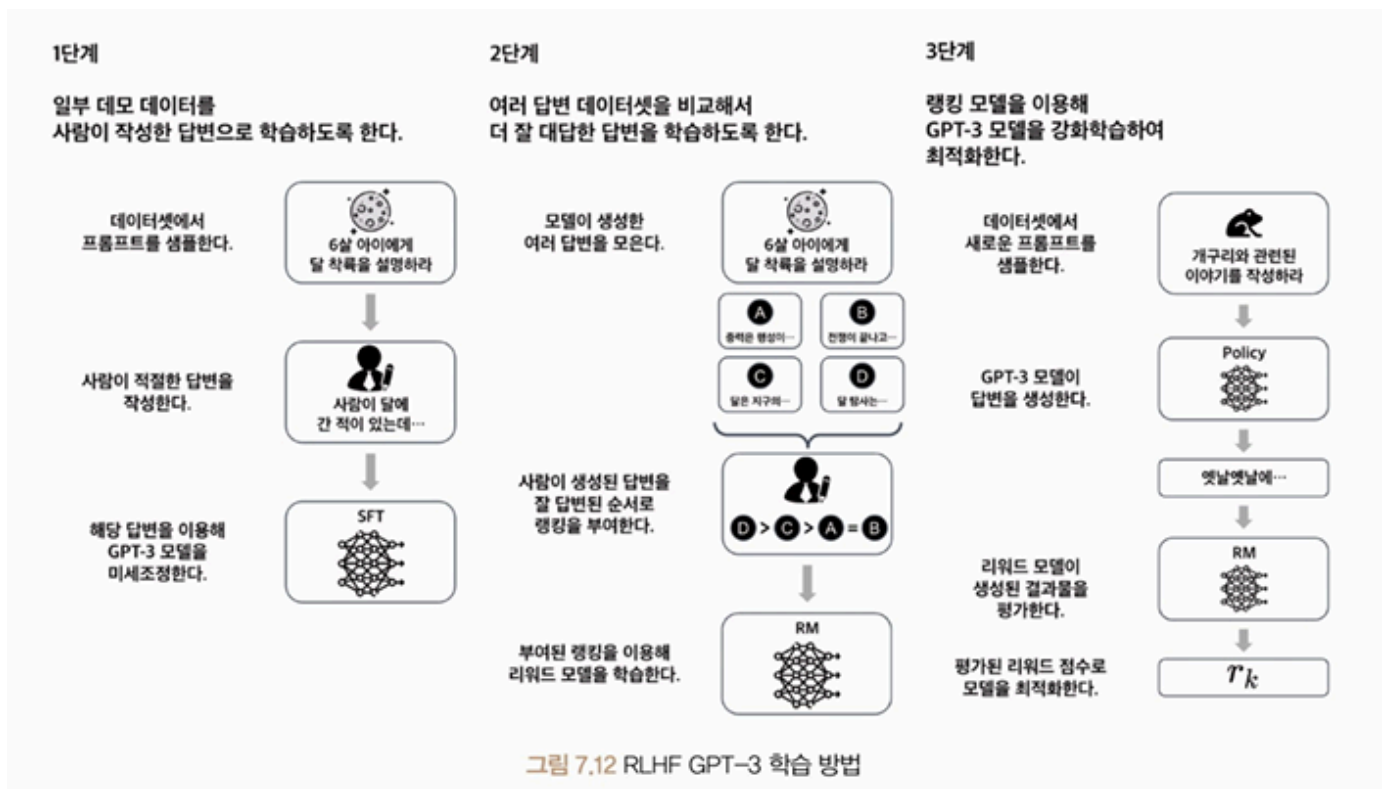
출력값

그림 7.11 GPT-3의 다운스트림 작업 프롬프트 예시

GPT-3는 새로운 도메인에서의 작업에 대해서도 높은 일반화 능력을 보이며, 기존 자연어 처리 분야에서는 이전의 기술적 한계를 극복하는 성과를 보여준다. 하지만 큰 모델이기에 매우 느리고 비용이 많이 들어 일반 사용자나 소규모 기업에서 활용하기 어려우며, 사전 학습된 데이터에 기반하여 작동하기 때문에 데이터가 편향돼 있거나 정확하지 않을 경우 오류가 발생할 수 있다.

GPT-3.5

GPT-3.5는 GPT-3의 구조를 그대로 따르면서도, 새로운 학습 방법인 RLHF를 도입해 모델의 매개변수 수를 줄이고 모델의 자연스러움을 높였다.



RLHF를 통해 학습한 모델은 자연스러운 문장을 생성할 수 있다. 하지만 학습 데이터양을 고려하면 전체 프롬프트에 사람이 답변을 다는 것은 현실적으로 어렵다.

GPT-4

GPT-4는 이전 모델들과는 달리, 텍스트 데이터뿐만 아니라 이미지 데이터도 인식 가능한 멀티모달 모델이다.

모델 실습

예제 7.9 문장 생성을 위한 GPT-2 모델 구조

```
from transformers import GPT2LMHeadModel

model = GPT2LMHeadModel.from_pretrained(pretrained_model_name_or_path="gpt2")

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("├", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("│   └", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("│   │   └", sssub_name)
```

출력 결과

```
config.json: 100% 665/665 [00:00<00:00, 21.7kB/s]model.safetensors: 100% 548M/548M [00:08<00:
Key          | Status      | |
-----+-----+---
h.{0...11}.attn.bias | UNEXPECTED | |
```

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you e
generation_config.json: 100% 124/124 [00:00<00:00, 13.7kB/s]transformer

```
└ wte
└ wpe
└ drop
└ h
  └ 0
    └ ln_1
    └ attn
    └ ln_2
    └ mlp
  └ 1
    └ ln_1
    └ attn
    └ ln_2
    └ mlp
  └ 2
    └ ln_1
    └ attn
    └ ln_2
    └ mlp
  └ 3
    └ ln_1
    └ attn
    └ ln_2
    └ mlp
  └ 4
    └ ln_1
    └ attn
    └ ln_2
    └ mlp
  └ 5
    └ ln_1
    └ attn
    └ ln_2
    └ mlp
  └ 6
    └ ln_1
    └ attn
    └ ln_2
    └ mlp
  └ 7
    └ ln_1
    └ attn
    └ ln_2
    └ mlp
  └ 8
    └ ln_1
    └ attn
```

```

| | L ln_2
| | L mlp
| L 9
| | L ln_1
| | L attn
| | L ln_2
| | L mlp
| L 10
| | L ln_1
| | L attn
| | L ln_2
| | L mlp
| L 11
| | L ln_1
| | L attn
| | L ln_2
| | L mlp
L ln_f
lm_head

```

예제 7.10 GPT-2를 이용한 문장 생성

```

from transformers import pipeline

generator = pipeline(task="text-generation", model="gpt2")
outputs = generator(
    text_inputs="Machine learning is",
    max_length=20,
    num_return_sequences=3,
    pad_token_id=generator.tokenizer.eos_token_id
)
print(outputs)

```

출력 결과

```

Loading weights: 100% 148/148 [00:00<00:00, 364.43it/s, Materializing param=transformer.wte.w
Key          | Status      | |
-----+-----+---
h.{0...11}.attn.bias | UNEXPECTED | |

Notes:
- UNEXPECTED      :can be ignored when loading from different task/architecture; not ok if you e
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 2.45kB/s]vocab.json: 1.04M/? [00:00<00:00
Both `max_new_tokens` (=256) and `max_length` (=20) seem to have been set. `max_new_tokens` wil
[{'generated_text': 'Machine learning is a good way to get information about your business. Bu

```

예제 7.11 CoLA 데이터셋 불러오기

```

import torch
from datasets import load_dataset

```

```

from transformers import AutoTokenizer
from torch.utils.data import DataLoader

def collator(batch, tokenizer, device):
    source, labels, texts = zip(*batch)
    tokenized = tokenizer(
        list(texts),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(labels, dtype=torch.long).to(device)
    return input_ids, attention_mask, labels

# — CoLA → datasets로 대체 —————
cola = load_dataset("nyu-mll/glue", "cola")

def to_tuple(item):
    return (None, item["label"], item["sentence"])

train_data = [to_tuple(item) for item in cola["train"]]
valid_data = [to_tuple(item) for item in cola["validation"]]
test_data = [to_tuple(item) for item in cola["test"]]
# —————

tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token

epochs = 3
batch_size = 16
device = "cuda" if torch.cuda.is_available() else "cpu"

train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    collate_fn=lambda x: collator(x, tokenizer, device),
    shuffle=True,
)
valid_dataloader = DataLoader(
    valid_data, batch_size=batch_size, collate_fn=lambda x: collator(x, tokenizer, device)
)
test_dataloader = DataLoader(
    test_data, batch_size=batch_size, collate_fn=lambda x: collator(x, tokenizer, device)
)

print("Train Dataset Length :", len(train_data))
print("Valid Dataset Length :", len(valid_data))
print("Test Dataset Length :", len(test_data))

```

출력 결과

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md: 35.3k/? [00:00<00:00, 2.24MB/s]Warning: You are sending unauthenticated requests to the Hugging Face Hub.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the Hugging Face Hub.
cola/train-000000-of-000001.parquet: 100% 251k/251k [00:01<00:00, 1.26MB/s]cola/validation-000000-of-000001.parquet: 100% 251k/251k [00:01<00:00, 1.26MB/s]
Valid Dataset Length : 1043
Test Dataset Length : 1063
```

예제 7.12 GPT-2 모델 설정

```
import torch
from torch import optim
from transformers import GPT2ForSequenceClassification

device = "cuda" if torch.cuda.is_available() else "cpu" # ← 이거 추가

model = GPT2ForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="gpt2",
    num_labels=2
).to(device)
model.config.pad_token_id = model.config.eos_token_id
optimizer = optim.Adam(model.parameters(), lr=5e-5)
```

예제 7.13 GPT-2 모델 학습 및 검증

```
import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss
        train_loss += loss.item()

    optimizer.zero_grad()
```



```

        loss.backward()
        optimizer.step()

    train_loss = train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss, val_accuracy = 0.0, 0.0

        for input_ids, attention_mask, labels in dataloader:
            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )
            logits = outputs.logits
            loss = outputs.loss

            logits = logits.detach().cpu().numpy()
            label_ids = labels.to("cpu").numpy()
            accuracy = calc_accuracy(logits, label_ids)

            val_loss += loss.item()
            val_accuracy += accuracy

        val_loss = val_loss/len(dataloader)
        val_accuracy = val_accuracy/len(dataloader)
        return val_loss, val_accuracy

best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Accur

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "../models/GPT2ForSequenceClassification.pt")
        print("Saved the model weights")

```

출력 결과

```

Epoch 1: Train Loss: 0.6178 Val Loss: 0.6245 Val Accuracy 0.6910
Saved the model weights
Epoch 2: Train Loss: 0.6100 Val Loss: 0.6175 Val Accuracy 0.6910
Saved the model weights
Epoch 3: Train Loss: 0.6096 Val Loss: 0.6173 Val Accuracy 0.6900
Saved the model weights

```

예측 7.14 모델 평가

```

model = GPT2ForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="gpt2",
    num_labels=2
).to(device)
model.config.pad_token_id = model.config.eos_token_id
model.load_state_dict(torch.load("../models/GPT2ForSequenceClassification.pt"))

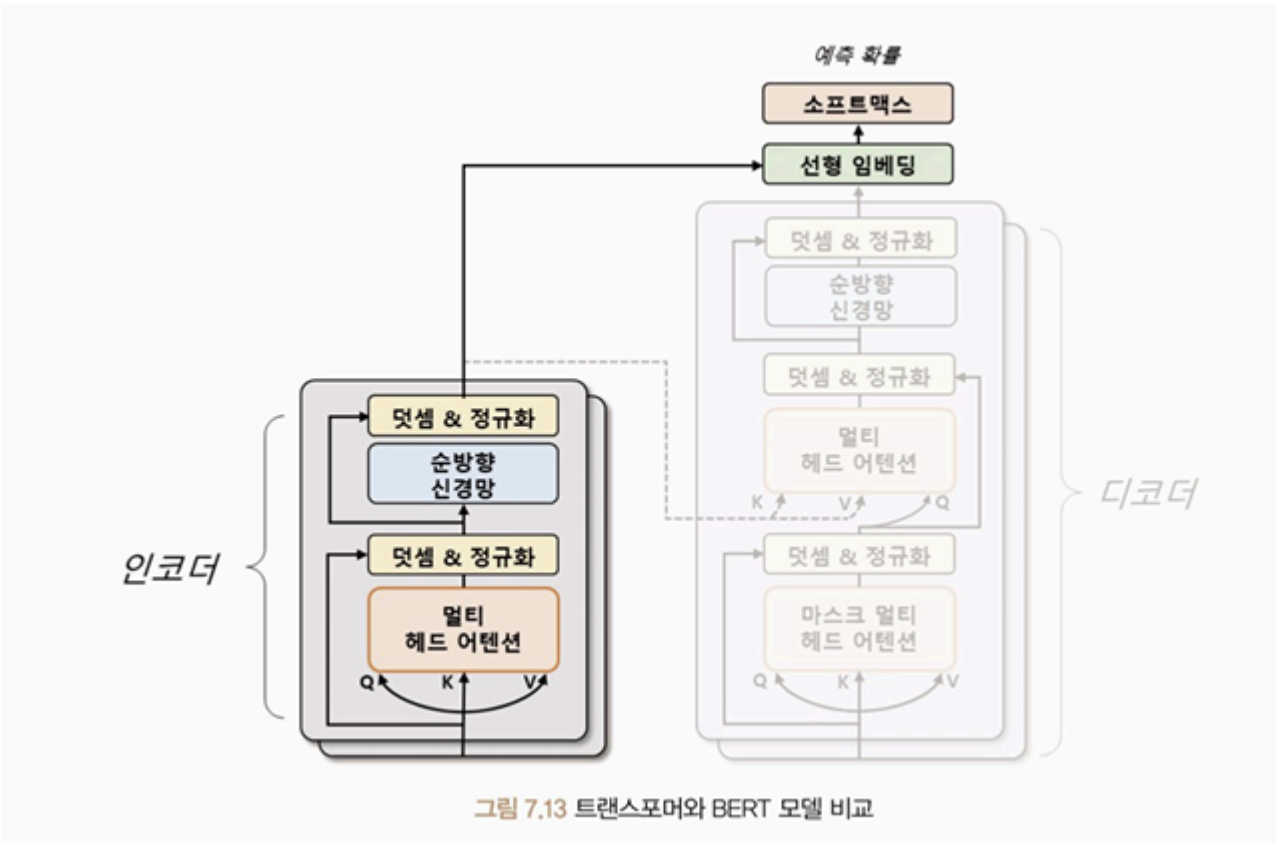
test_loss, test_accuracy = evaluation(model, test_dataloader)
print(f"Test Loss : {test_loss:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")

```

BERT

BERT: 2018년 구글에서 발표한 언어 모델로 트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델

양방향 인코더: 입력 시퀀스를 양쪽 방향에서 처리하여 이전과 이후의 단어를 모두 참조하며 단어의 의미와 문맥을 파악한다.



사전 학습 방법

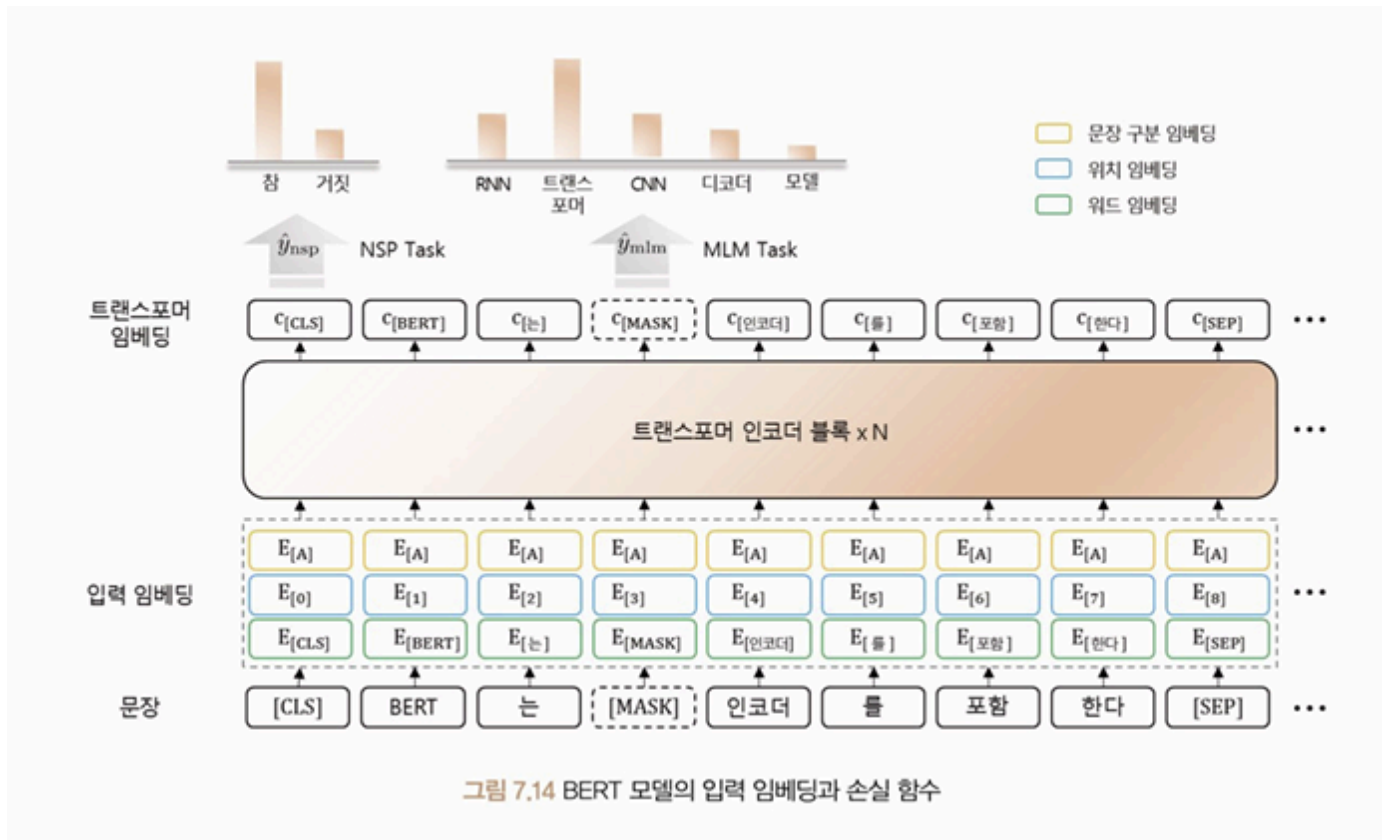
마스킹된 언어 모델링: 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식

다음 문장 예측: 두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업

[CLS] 토큰 : 입력 문장의 시작 부분에 추가되는 초크

[SEP] 토큰 : 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용되는 토큰

[MASK] 토큰 : 입력 문장 내에서 임의로 선택된 단어를 가리키는 특별한 토큰



모델 실습

예제 7.15 네이버 영화 리뷰 데이터 불러오기

```
import numpy as np
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
df = pd.DataFrame(corpus.test).sample(20000, random_state=42)

train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(train.head(5).to_markdown())
print(f"Training Data Size : {len(train)}")
```

```
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

출력 결과

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을
손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다.
해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 **description** 을 참고,
해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```
# Description
Author : e9t@github
Repository : https://github.com/e9t/nsmc
References : www.lucypark.kr/docs/2015-pyconkr/#39
```

```
Naver sentiment movie corpus v1.0
This is a movie review dataset in the Korean language.
Reviews were scraped from Naver Movies.
```

```
The dataset construction is based on the method noted in
[Large movie review dataset][^1] from Maas et al., 2011.
```

```
[^1]: http://ai.stanford.edu/~amaas/data/sentiment/
```

```
# License
CC0 1.0 Universal (CC0 1.0) Public Domain Dedication
Details in https://creativecommons.org/publicdomain/zero/1.0/
```

```
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_train.txt
```

```
[Korpora] Corpus `nsmc` is already installed at /root/Korpora/nsmc/ratings_test.txt
```

	text	label
26891	역시 코믹액션은 성룡, 홍금보, 원표 삼인방이 최고지!!	1
25024	점수 후하게 줘야겠네 별 반개~	0
11666	오랜만에 느낄수 있는 [감독] 구타육구.	0
40303	본지는 좀 똥지만 극장서 돈주고 본게 아직까지 아까운 영화	0
18010	징키스칸이란 소재를 가지고 이것밖에 못만드냐	0

```
Training Data Size : 12000
```

```
Validation Data Size : 4000
```

```
Testing Data Size : 4000
```

```
/usr/local/lib/python3.12/dist-packages/numpy/_core/fromnumeric.py:54: FutureWarning: 'DataFrame
return bound(*args, **kws)
```

예제 7.16 BERT 입력 텐서 생성

```
import torch
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
```

```

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_datalodader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 5
batch_size = 32
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    do_lower_case=False
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_datalodader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_datalodader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_datalodader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])

```

출력 결과

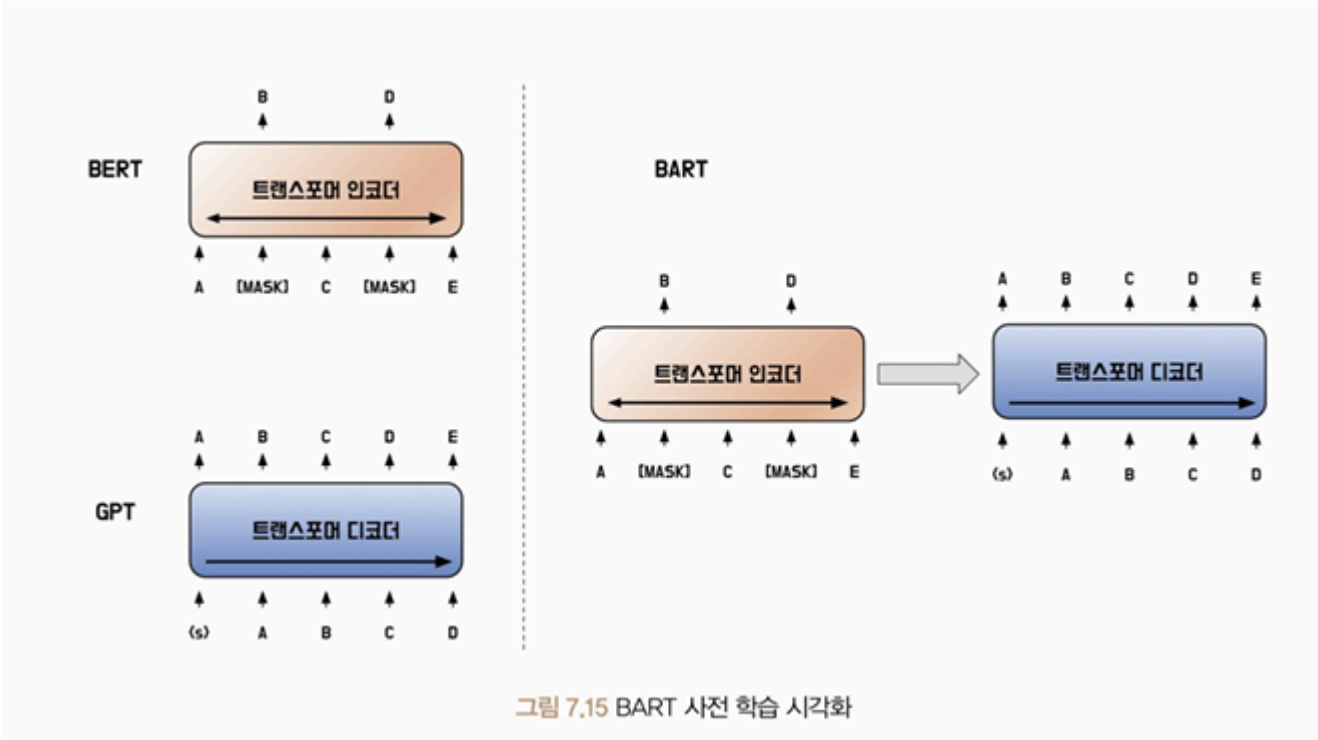
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.com/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable authentication.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable authentication.
tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 1.69kB/s]vocab.txt: 996k/? [00:00<00:00,
117, 9992, 40032, 30005, 117, 9612, 37824, 9410, 12030,
42337, 10739, 83491, 12508, 106, 106, 102, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0,

```


잡음이 없는 원본 데이터를 복원하도록 학습하는 방식으로 수행된다.

BERT 인코더는 입력 문장에서 일부 단어를 무작위로 마스킹해 처리하고, 마스킹된 단어를 맞추도록 학습한다.



BART는 사전 학습 시 노이즈 제거 오토인코더를 사용하므로 입력 문장에 임의로 노이즈를 추가하고 원래 문장을 복원하도록 학습한다.

노이즈가 추가된 텍스트를 인코더에 입력하고 원본 텍스트를 디코더에 입력해 디코더가 원본 텍스트를 생성할 수 있게 학습하는 방식이다.

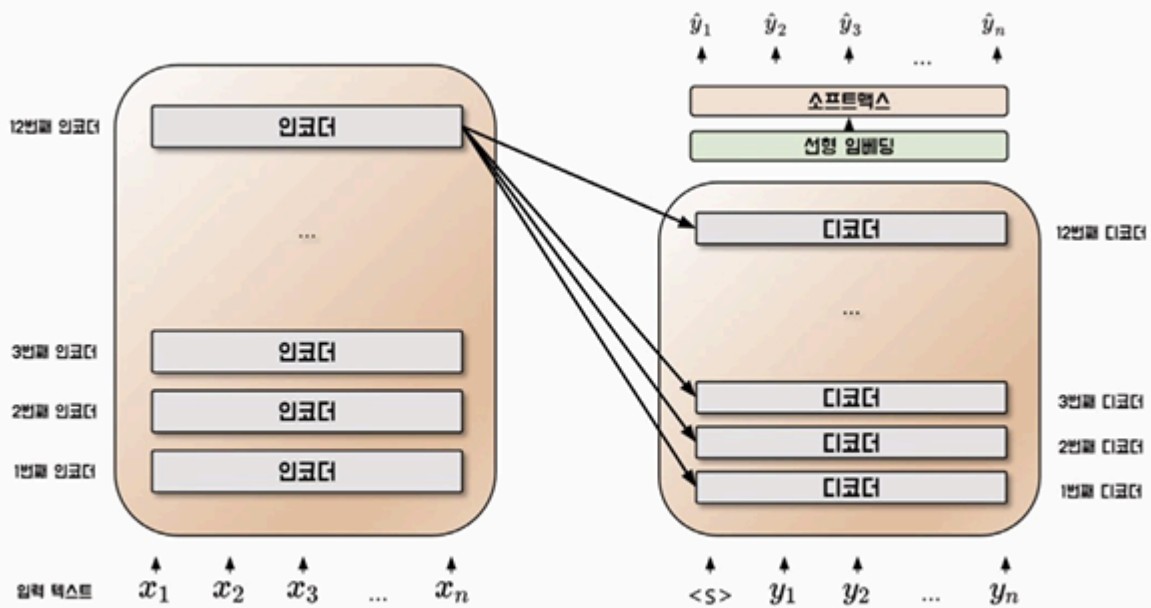


그림 7.16 BART 모델 구조

사전 학습 방법

원본 문장

그는 정문으로 발을 옮겼다. 그의 아내는 멀어지는 그를
바라보고 있었다. 그녀는 눈물을 흘렸다. 그도
마찬가지였다.

노이즈 기법

토큰 마스킹

그는 _ 발을 옮겼다. _ 아내는 멀어지는 그를 _ 있었다.
그녀는 눈물을 _ 그도 _ 었다.

토큰 삭제

그는 발을 옮겼다. 아내는 멀어지는 그를 있었다.
그녀는 눈물을 , 그도 었다.

문장 순열

그녀는 눈물을 흘렸다. 그는 정문으로 발을 옮겼다.
그의 아내는 멀어지는 그를 바라보고 있었다. 그도 마찬가지였다.

문서 회전

그를 바라보고 있었다. 그녀는 눈물을 흘렸다. 그도 마찬가지였다.
그는 정문으로 발을 옮겼다. 아내는 멀어지는

텍스트 채우기

그는 옮겼다. 아내는 멀어지는 그를 ,
그녀는 눈물을 흘렸다. 그도 ,

그림 7.17 BART 노이즈 기법

토큰 마스킹: BERT에서 사용한 MLM과 동일한 기법으로, 입력 문장의 일부 토큰을 마스크

토큰으로 치환하는 방법

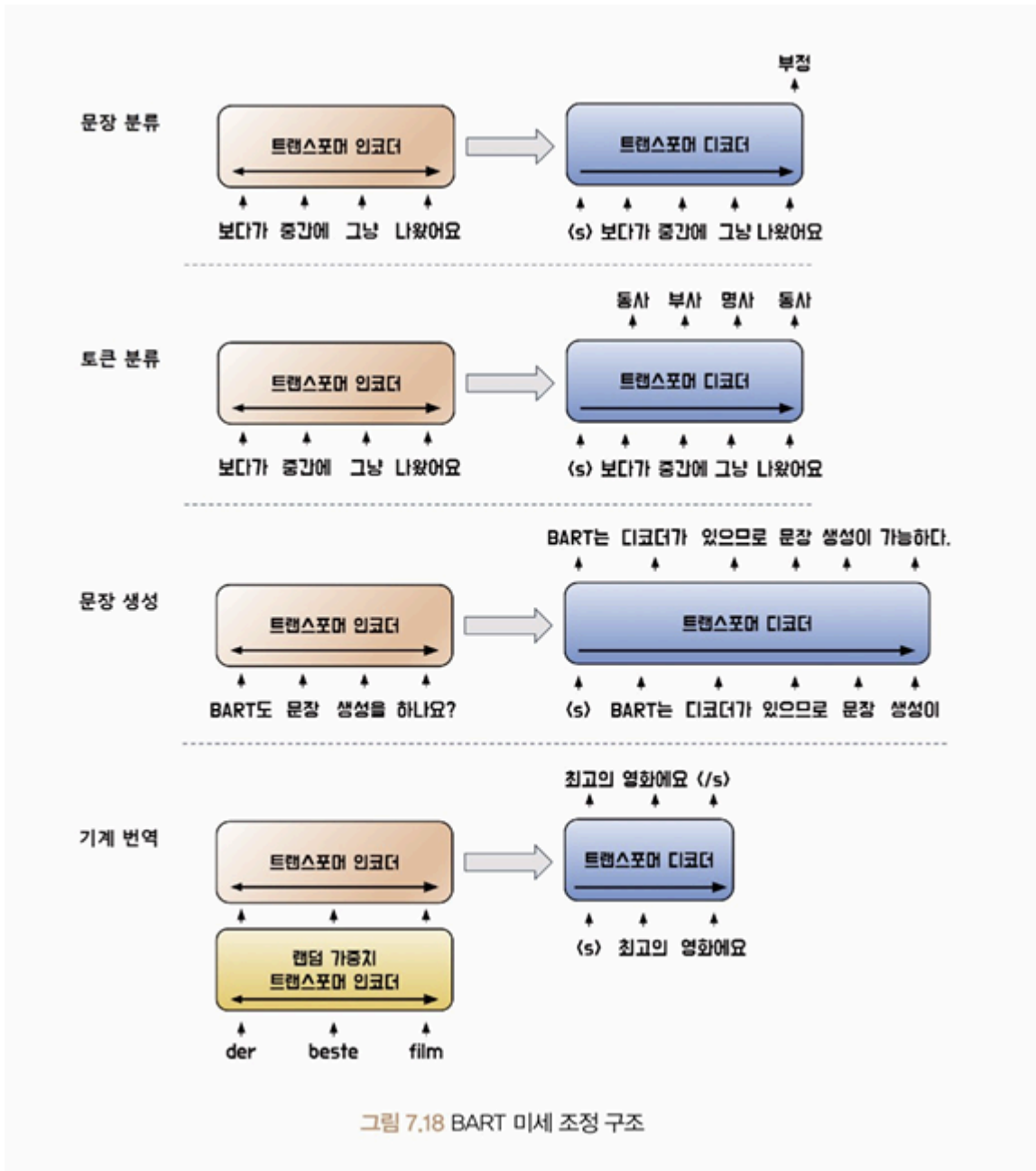
토큰 삭제: 입력 문장의 일부 토큰을 치환하는 것이 아니라, 삭제하는 방법

문장 순열: 마침표를 기준으로 문장을 나눈 뒤, 문장의 순서를 섞는 방법

문서 회전: 임의의 토큰으로 문서가 시작하도록 함

텍스트 채우기: 몇 개의 토큰을 하나의 구간으로 묶고 일부 구간을 마스크 토큰으로 대체함

미세 조정 방법



BART는 트랜스포머 디코더를 사용하기 때문에 BERT가 해결하지 못했던 문장 생성 작업을 수행할 수 있다. 특히 입력값을 조작해 출력을 생성하는 추상적 질의응답과 문장 요약과 같은 작업에 적합하다.

모델 실습

예제 7.18 뉴스 요약 데이터셋 불러오기

```
import numpy as np
from datasets import load_dataset

news = load_dataset("argilla/news-summary", split="test")
df = news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(f"Source News : {train.text.iloc[0][:200]}")
print(f"Summarization : {train.prediction.iloc[0][:50]}")
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

출력 결과

```
README.md: 2.02k/? [00:00<00:00, 200kB/s]data/train-000000-of-00001-ebc48879f34571(...): 100% 1
Summarization : Putin says had useful interaction with Trump at Vi
Training Data Size : 3000
Validation Data Size : 1000
Testing Data Size : 1000
/usr/local/lib/python3.12/dist-packages/numpy/_core/fromnumeric.py:54: FutureWarning: 'DataFra
return bound(*args, **kwargs)
```

예제 7.19 BART 입력 텐서 생성

```
import torch
from transformers import BartTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from torch.nn.utils.rnn import pad_sequence

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
```

```

        return_tensors="pt",
        max_length=1024
    )
    labels = []
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    for target in data.prediction:
        labels.append(tokenizer.encode(target, return_tensors="pt").squeeze())
    labels = pad_sequence(labels, batch_first=True, padding_value=-100).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_datalodader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 5
batch_size = 8
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = BartTokenizer.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_datalodader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_datalodader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_datalodader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])

```

출력 결과

```

vocab.json: 899k/? [00:00<00:00, 13.6MB/s]merges.txt: 456k/? [00:00<00:00, 7.73MB/s]tokeniz
3564,      2,  -100,  -100,  -100,  -100,  -100,  -100,  -100,  -100,  -100,
-100,  -100,  -100,  -100,  -100,  -100,  -100,  -100,  -100,  -100],
device='cuda:0'))

```

예제 7.20 BART 모델 선언

```

from torch import optim
from transformers import BartForConditionalGeneration

model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
)

```

```

).to(device)
optimizer = optim.AdamW(model.parameters(), lr=5e-5, eps=1e-8)

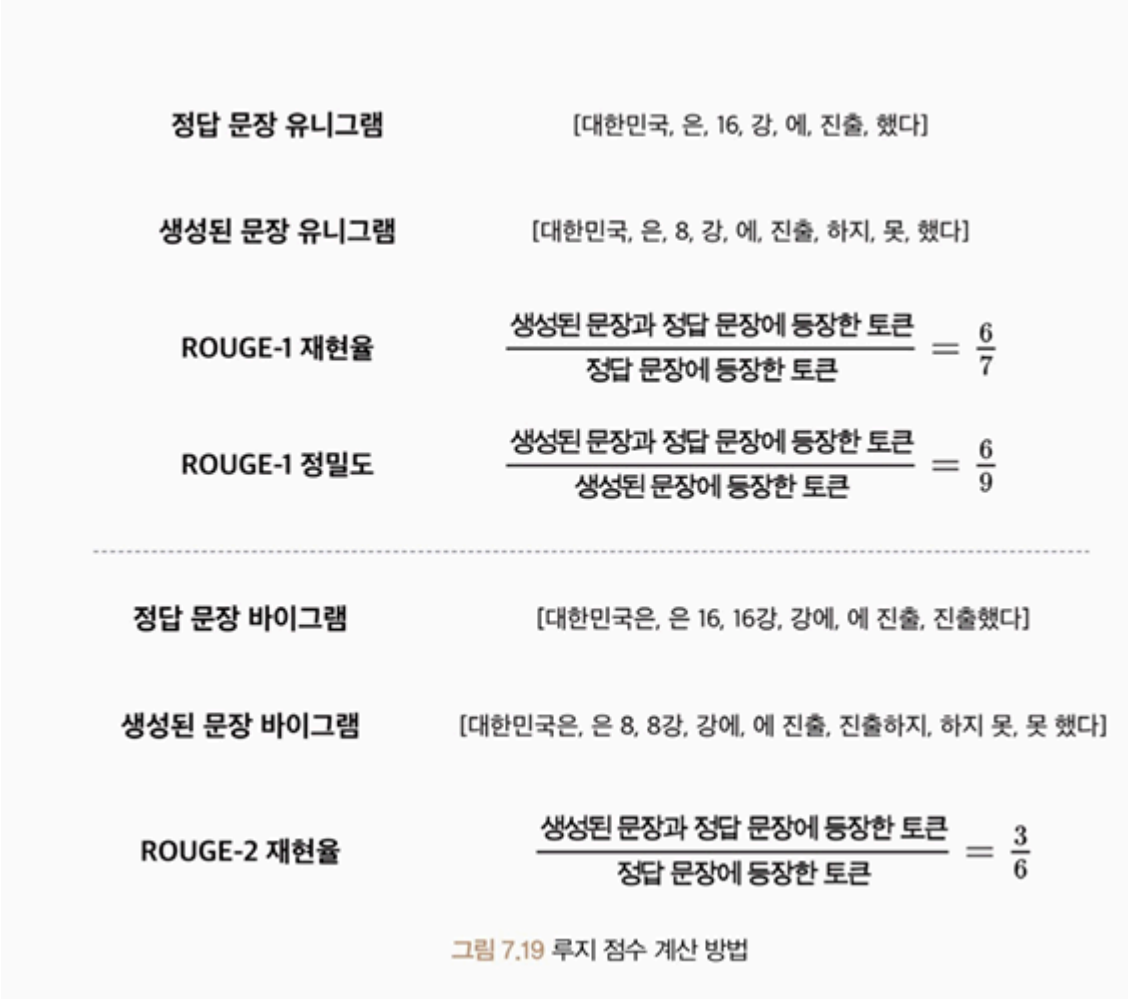
```

출력 결과

```

config.json: 1.72k/? [00:00<00:00, 48.3kB/s]model.safetensors: 100% 558M/558M [00:04<00:00,

```



예제 7.21 BART 모델 학습 및 평가

```

import numpy as np
import evaluate

def calc_rouge(preds, labels):
    preds = preds.argmax(axis=-1)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

```

```

rouge2 = rouge_score.compute(
    predictions=decoded_preds,
    references=decoded_labels
)
return rouge2["rouge2"]

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss
        train_loss += loss.item()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    train_loss = train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss, val_rouge = 0.0, 0.0

        for input_ids, attention_mask, labels in dataloader:
            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )
            logits = outputs.logits
            loss = outputs.loss

            logits = logits.detach().cpu().numpy()
            label_ids = labels.to("cpu").numpy()
            rouge = calc_rouge(logits, label_ids)

            val_loss += loss.item()
            val_rouge += rouge

        val_loss = val_loss / len(dataloader)
        val_rouge = val_rouge / len(dataloader)
        return val_loss, val_rouge

rouge_score = evaluate.load("rouge", tokenizer=tokenizer)
best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Rouge

```

```

if val_loss < best_loss:
    best_loss = val_loss
    torch.save(model.state_dict(), "../models/BartForConditionalGeneration.pt")
    print("Saved the model weights")

```

출력 결과

```

Downloading builder script:
6.14k/? [00:00<00:00, 151kB/s]
Epoch 1: Train Loss: 2.1932 Val Loss: 1.8763 Val Rouge 0.2359
Saved the model weights
Epoch 2: Train Loss: 1.6207 Val Loss: 1.8891 Val Rouge 0.2520
Epoch 3: Train Loss: 1.2527 Val Loss: 1.9453 Val Rouge 0.2268
Epoch 4: Train Loss: 0.9685 Val Loss: 2.0856 Val Rouge 0.2469
Epoch 5: Train Loss: 0.7413 Val Loss: 2.2326 Val Rouge 0.2322

```

예제 7.22 BART 모델 평가

```

model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
).to(device)
model.load_state_dict(torch.load("../models/BartForConditionalGeneration.pt"))

test_loss, test_rouge_score = evaluation(model, test_dataloader)
print(f"Test Loss : {test_loss:.4f}")
print(f"Test ROUGE-2 Score : {test_rouge_score:.4f}")

```

출력 결과

```

Loading weights: 100%
259/259 [00:00<00:00, 707.22it/s, Materializing param=model.shared.weight]
Test Loss : 1.8355
Test ROUGE-2 Score : 0.2445

```

예제 7.23 문장 요약문 비교

```

model.eval()

for index in range(5):
    news_text = test.text.iloc[index]
    summarization = test.prediction.iloc[index]

    inputs = tokenizer(
        news_text,
        return_tensors="pt",
        truncation=True,
        max_length=512
    ).to(device)

```

```

with torch.no_grad():
    output_ids = model.generate(
        **inputs,
        max_length=54,
        min_length=10,
        do_sample=False
    )

    predicted_summarization = tokenizer.decode(output_ids[0], skip_special_tokens=True)
    print(f"정답 요약문 : {summarization}")
    print(f"모델 요약문 : {predicted_summarization}\n")

```

정답 요약문 : Clinton leads Trump by 4 points in Washington Post: ABC News poll

모델 요약문 : Clinton leads Trump by 4 points: Washington Post-ABC News poll

정답 요약문 : Democrats question independence of Trump Supreme Court nominee

모델 요약문 : U.S. senators question Gorsuch's independence

정답 요약문 : In push for Yemen aid, U.S. warned Saudis of threats in Congress

모델 요약문 : U.S. warns Saudi Arabia over humanitarian situation in Yemen

정답 요약문 : Romanian ruling party leader investigated over 'criminal group'

모델 요약문 : Romanian prosecutors charge party leader with 'criminal group'

정답 요약문 : Billionaire environmental activist Tom Steyer endorses Clinton

모델 요약문 : Environmental activist Steyer backs Clinton for U.S. president

ELECTRA

ELECTRA: 2020년 구글에서 발표한 트랜스포머 기반 모델

ELECTRA는 입력을 마스킹하는 대신 생성자와 판별자를 사용해 사전 학습을 수행한다.

ELECTRA의 사전 학습 접근 방식은 변환기 모델인 생성자와 판별자를 학습하므로 생성적 적대 신경망(GAN)과 유사한 방법으로 학습이 수행된다.

생성 모델은 실제 데이터와 비슷하게 토큰을 생성해 다른 토큰으로 대체하고 판별 모델이 생성 모델이 만든 데이터와 실제 데이터를 입력받아 어떤 데이터가 실제 데이터인지, 아니면 생성된 데이터인지 구분한다.

ELECTRA는 GAN을 사용해 학습하므로 이전 언어 모델과 비교하여 더 효율적인 학습이 가능하다. 덕분에 대규모 데이터셋에서 모델을 더 빠르게 학습할 수 있다.

그리고 BERT와 같은 모델과 비교하면 모델의 매개변수가 더 적다. 모델 크기는 줄어들어 모델을 더 빠르게 실행할 수 있으며, 더 적은 메모리를 사용한다.

사전 학습 방법

ELECTRA의 생성자 모델과 판별자 모델은 모두 트랜스포머 인코더 구조를 따른다. 생성자 모델은 입력 문장의 일부 토큰을 마스크 처리하고 마스크 처리된 토큰이 원래 어떤 토큰이었는지 예측하며 학습한다.

반면에 판별자 모델은 각 입력 토큰이 원본 문장의 토큰인지 생성자 모델로 인해 바뀐 토큰인지 맞히며 연습한다. 이런 학습 방법을 RTD라고 한다.



그림 7.20 ELECTRA 모델 학습 방법

모델 실습

예제 7.24 네이버 영화 리뷰 데이터셋 전처리

```
import torch
from transformers import ElectraTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader
```



```
epochs = 5
batch_size = 32
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = ElectraTokenizer.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    do_lower_case=False,
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])
```

```
(tensor([ 2, 6511, 14347, 4087, 4665, 4112, 2924, 4806, 16, 3809,  
         4309, 4275, 16, 3201, 4376, 2891, 4139, 4212, 4007, 6557,  
        4200, 5, 5, 3, 0, 0, 0, 0, 0, 0,  
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
         0], device='cuda:0'), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
        0], device='cuda:0'), tensor(1, device='cuda:0'))
```

```
from torch import optim
from transformers import ElectraForSequenceClassification

model = ElectraForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    num_labels=2
).to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

출력 결과

```
config.json: 100% 467/467 [00:00<00:00, 26.2kB/s]pytorch_model.bin: 100% 452M/452M [00:09<00:00:00, 10.8MB/s]
Key | Status |
```

Key	Status
electra.embeddings.position_ids	UNEXPECTED
discriminator_predictions.dense_prediction.weight	UNEXPECTED
discriminator_predictions.dense.weight	UNEXPECTED
discriminator_predictions.dense_prediction.bias	UNEXPECTED
discriminator_predictions.dense.bias	UNEXPECTED
classifier.out_proj.weight	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.bias	MISSING
classifier.dense.bias	MISSING

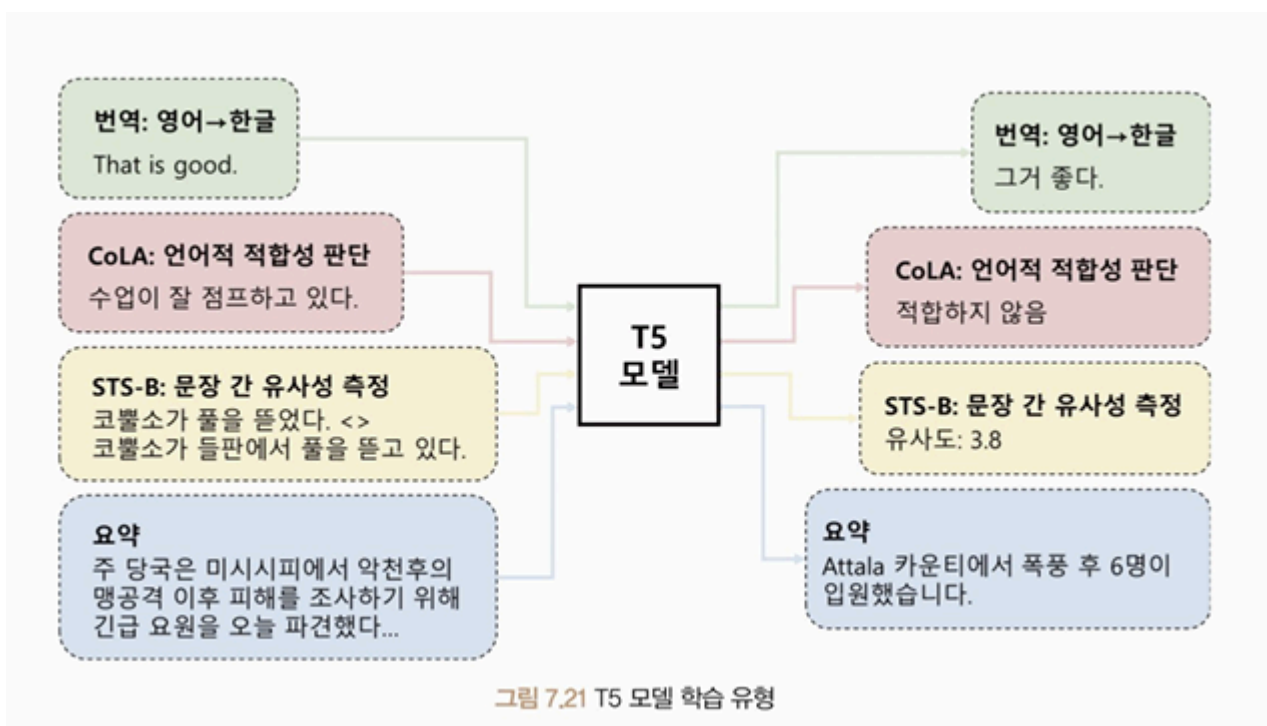
Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you e
- MISSING :those params were newly initialized because missing from the checkpoint. Consider

T5

T5: 2019년 구글에서 발표한 자연어 처리 분야 딥러닝 모델로 트랜스포머 구조를 기반으로 한다.

T5는 입력과 출력을 모두 토큰 시퀀스로 처리하는 텍스트-텍스트 구조다. 따라서 입력과 출력 형태를 자유롭게 다룰 수 있으며, 모델 구조상 유연성과 확장성이 뛰어나기 때문에 새로운 자연어 처리 작업에서도 쉽게 적용할 수 있다.



모델 실습

예제 7.26 뉴스 요약 데이터셋 불러오기

```
import numpy as np
from datasets import load_dataset

news = load_dataset("argilla/news-summary", split="test")
df = news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["text"] = "summarize: " + df["text"]
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(f"Source News : {train.text.iloc[0][:200]}")
print(f"Summarization : {train.prediction.iloc[0][:50]}")
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

출력 결과

```
Source News : summarize: DANANG, Vietnam (Reuters) - Russian President Vladimir Putin said on
Summarization : Putin says had useful interaction with Trump at Vi
Training Data Size : 3000
Validation Data Size : 1000
Testing Data Size : 1000
/usr/local/lib/python3.12/dist-packages/numpy/_core/fromnumeric.py:54: FutureWarning: 'DataFra
return bound(*args, **kws)
```

예제 7.27 뉴스 요약 데이터셋 전처리

```
import torch
from transformers import T5Tokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from torch.nn.utils.rnn import pad_sequence

def make_dataset(data, tokenizer, device):
    source = tokenizer(
        text=data.text.tolist(),
        padding="max_length",
        max_length=128,
        pad_to_max_length=True,
        truncation=True,
        return_tensors="pt"
    )

    target = tokenizer(
        text=data.prediction.tolist(),
        padding="max_length",
        max_length=128,
```

```

        pad_to_max_length=True,
        truncation=True,
        return_tensors="pt"
    )

    source_ids = source["input_ids"].squeeze().to(device)
    source_mask = source["attention_mask"].squeeze().to(device)
    target_ids = target["input_ids"].squeeze().to(device)
    target_mask = target["attention_mask"].squeeze().to(device)
    return TensorDataset(source_ids, source_mask, target_ids, target_mask)

def get_datalodader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 5
batch_size = 8
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = T5Tokenizer.from_pretrained(
    pretrained_model_name_or_path="t5-small"
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_datalodader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_datalodader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_datalodader(test_dataset, SequentialSampler, batch_size)

print(next(iter(train_dataloader)))
print(tokenizer.convert_ids_to_tokens(21603))
print(tokenizer.convert_ids_to_tokens(10))

```

출력 결과

```

tokenizer_config.json: 2.32k/? [00:00<00:00, 121kB/s]spiece.model: 100% 792k/792k [00:00<00:
[21603, 10, 41, ..., 1104, 11893, 1],
[21603, 10, 3, ..., 11106, 681, 1],
...,
[21603, 10, 549, ..., 133, 43, 1],
[21603, 10, 549, ..., 868, 5869, 1],
[21603, 10, 549, ..., 12, 617, 1]], device='cuda:0'), tensor([[1, 1, 1,
[1, 1, 1, ..., 1, 1, 1],
[1, 1, 1, ..., 1, 1, 1],
...,
[1, 1, 1, ..., 1, 1, 1],
[1, 1, 1, ..., 1, 1, 1],
[1, 1, 1, ..., 1, 1, 1]], device='cuda:0'), tensor([[ 3, 476, 4902, ..., 0
[11543, 2689, 10, ..., 0, 0, 0],
[ 2523, 845, 73, ..., 0, 0, 0],
...,
[ 2759, 2523, 1082, ..., 0, 0, 0],

```

```

[ 412,    5,  134, ...,  0,    0,    0],
[ 2523,  870,    7, ...,  0,    0,    0]], device='cuda:0'), tensor([[1, 1, 1,
[1, 1, 1, ..., 0, 0, 0],
[1, 1, 1, ..., 0, 0, 0],
...,
[1, 1, 1, ..., 0, 0, 0],
[1, 1, 1, ..., 0, 0, 0],
[1, 1, 1, ..., 0, 0, 0]], device='cuda:0')]
__summary
:

```

예제 7.28 T5 모델 선언

```

from torch import optim
from transformers import T5ForConditionalGeneration

model = T5ForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="t5-small",
).to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)

```

출력 결과

```

config.json: 1.21k/? [00:00<00:00, 95.3kB/s]model.safetensors: 100% 242M/242M [00:04<00:00,

```



예제 7.29 T5 모델 학습 및 평가

```

import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for source_ids, source_mask, target_ids, target_mask in dataloader:
        decoder_input_ids = target_ids[:, :-1].contiguous()
        labels = target_ids[:, 1:].clone().detach()
        labels[target_ids[:, 1:] == tokenizer.pad_token_id] = -100

        outputs = model(
            input_ids=source_ids,
            attention_mask=source_mask,
            decoder_input_ids=decoder_input_ids,

```

```

        labels=labels,
    )

    loss = outputs.loss
    train_loss += loss.item()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

train_loss = train_loss / len(dataloader)
return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss = 0.0

        for source_ids, source_mask, target_ids, target_mask in dataloader:
            decoder_input_ids = target_ids[:, :-1].contiguous()
            labels = target_ids[:, 1:].clone().detach()
            labels[target_ids[:, 1:] == tokenizer.pad_token_id] = -100

            outputs = model(
                input_ids=source_ids,
                attention_mask=source_mask,
                decoder_input_ids=decoder_input_ids,
                labels=labels,
            )

            loss = outputs.loss
            val_loss += loss.item()

        val_loss = val_loss / len(dataloader)
        return val_loss

best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "../models/T5ForConditionalGeneration.pt")
        print("Saved the model weights")

```

출력 결과

```

Epoch 1: Train Loss: 4.3146 Val Loss: 3.3345
Saved the model weights
Epoch 2: Train Loss: 3.4353 Val Loss: 2.9203
Saved the model weights
Epoch 3: Train Loss: 3.1559 Val Loss: 2.7686

```

```
Saved the model weights
Epoch 4: Train Loss: 2.9810 Val Loss: 2.6788
Saved the model weights
Epoch 5: Train Loss: 2.8951 Val Loss: 2.6126
Saved the model weights
```

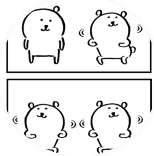
예제 7.30 T5 생성 모델 테스트

```
model.eval()
with torch.no_grad():
    for source_ids, source_mask, target_ids, target_mask in test_dataloader:
        generated_ids = model.generate(
            input_ids=source_ids,
            attention_mask=source_mask,
            max_length=128,
            num_beams=3,
            repetition_penalty=2.5,
            length_penalty=1.0,
            early_stopping=True,
        )

        for generated, target in zip(generated_ids, target_ids):
            pred = tokenizer.decode(
                generated, skip_special_tokens=True, clean_up_tokenization_spaces=True
            )
            actual = tokenizer.decode(
                target, skip_special_tokens=True, clean_up_tokenization_spaces=True
            )
            print("Generated Headline Text:", pred)
            print("Actual Headline Text    :", actual)
        break
```

출력 결과

```
Generated Headline Text: Clinton leads Trump by 4 percentage points in four-war race for Nov.
Actual Headline Text    : Clinton leads Trump by 4 points in Washington Post: ABC News poll
Generated Headline Text: U.S. senators sharpen potential line of attack against Gorsuch's nomi
Actual Headline Text    : Democrats question independence of Trump Supreme Court nominee
Generated Headline Text: U.S. warns Saudi Arabia over humanitarian situation in Yemen could co
Actual Headline Text    : In push for Yemen aid, U.S. warned Saudis of threats in Congress
Generated Headline Text: Romanian anti-corruption prosecutors open investigation into Liviu Dr
Actual Headline Text    : Romanian ruling party leader investigated over 'criminal group'
Generated Headline Text: environmental activist endorsed Hillary Clinton for U.S. president
Actual Headline Text    : Billionaire environmental activist Tom Steyer endorses Clinton
Generated Headline Text: tv presenter delivers news of Pyongyang nuclear test with her usual g
Actual Headline Text    : Voice of triumph or doom: North Korean presenter back in limelight fo
Generated Headline Text: Delson Guarate, Yon Goicoechea among nearly 400 jailed anti-Maduro ac
Actual Headline Text    : Venezuela frees two anti-Maduro activists; scores still jailed
Generated Headline Text: House Majority Leader says he still troubled by Clinton email server
Actual Headline Text    : House No. 2 Republican says still questions Clinton's judgment in ema
```



카사바

안녕하세요 :)



이전 포스트

Week11_예습과제

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate